



**RAJALAKSHMI
ENGINEERING COLLEGE**

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

DEPARTMENT OF INFORMATION TECHNOLOGY LAB MANUAL

CS23432 – Software Construction

(REGULATION 2023)

**RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015**

Name: AKASH K R

Register No: 241001010

Year / Branch / Section: II - IT

Semester: IV

Academic Year: 2025-2026

INDEX

S.No	Date	Title
1.	21/01/26	Azure Devops Environment Setup.
2.	28/01/26	Azure Devops Project Setup and User Story Management.
3.	04/02/26	Setting Up Epics, Features, And User Stories for Project Planning.
4.	11/02/26	Sprint Planning.
5.	18/02/26	Poker Estimation.
6.	25/02/26	Designing Class and Sequence Diagrams for Project Architecture.
7.	11/03/26	Designing Architectural and ER Diagrams for Project Structure.
8.	18/03/26	Testing – Test Plans and Test Cases.
9.	25/03/26	Load Testing and Pipelines.
10.	08/04/26	GitHub: Project Structure & Naming Conventions.

EXPNO: 1

Create Epic, Features, User Stories, Task

AZURE DEVOPS ENVIRONMENT SETUP

Aim:

To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

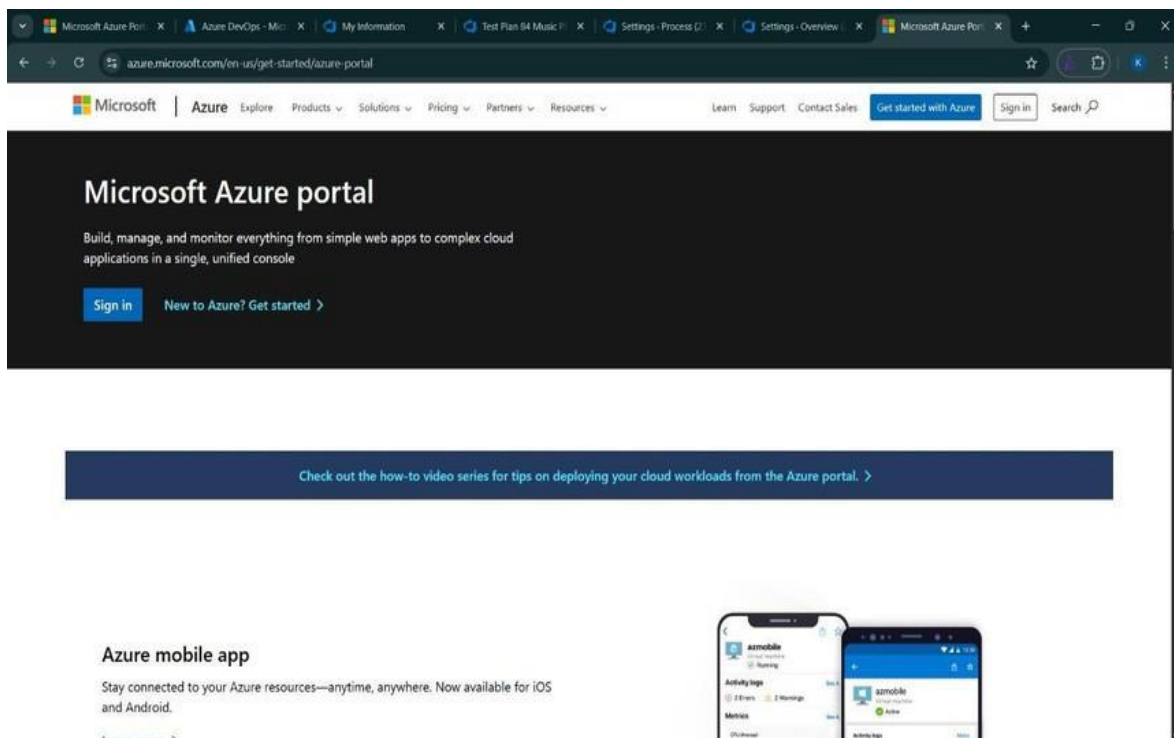
INSTALLATION:

1. Open your web browser and go to the Azure website:

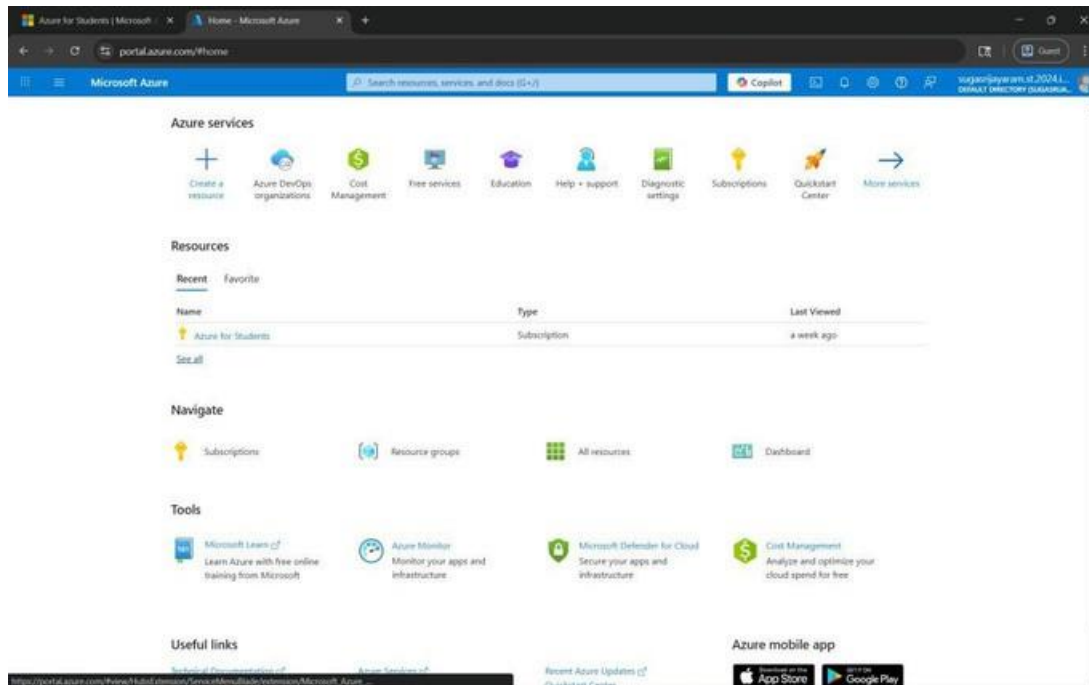
<https://azure.microsoft.com/en-us/get-started/azure-portal>.

Sign in using your Microsoft account credentials.

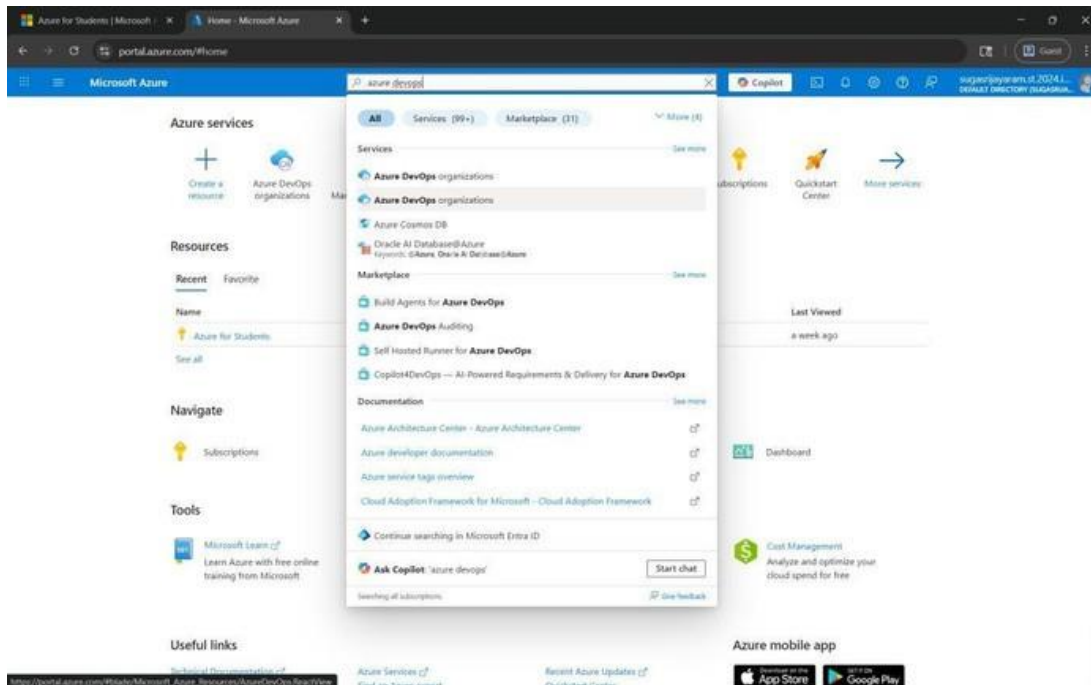
If you don't have a Microsoft account, you can create one here: <https://signup.live.com/?lic=1>



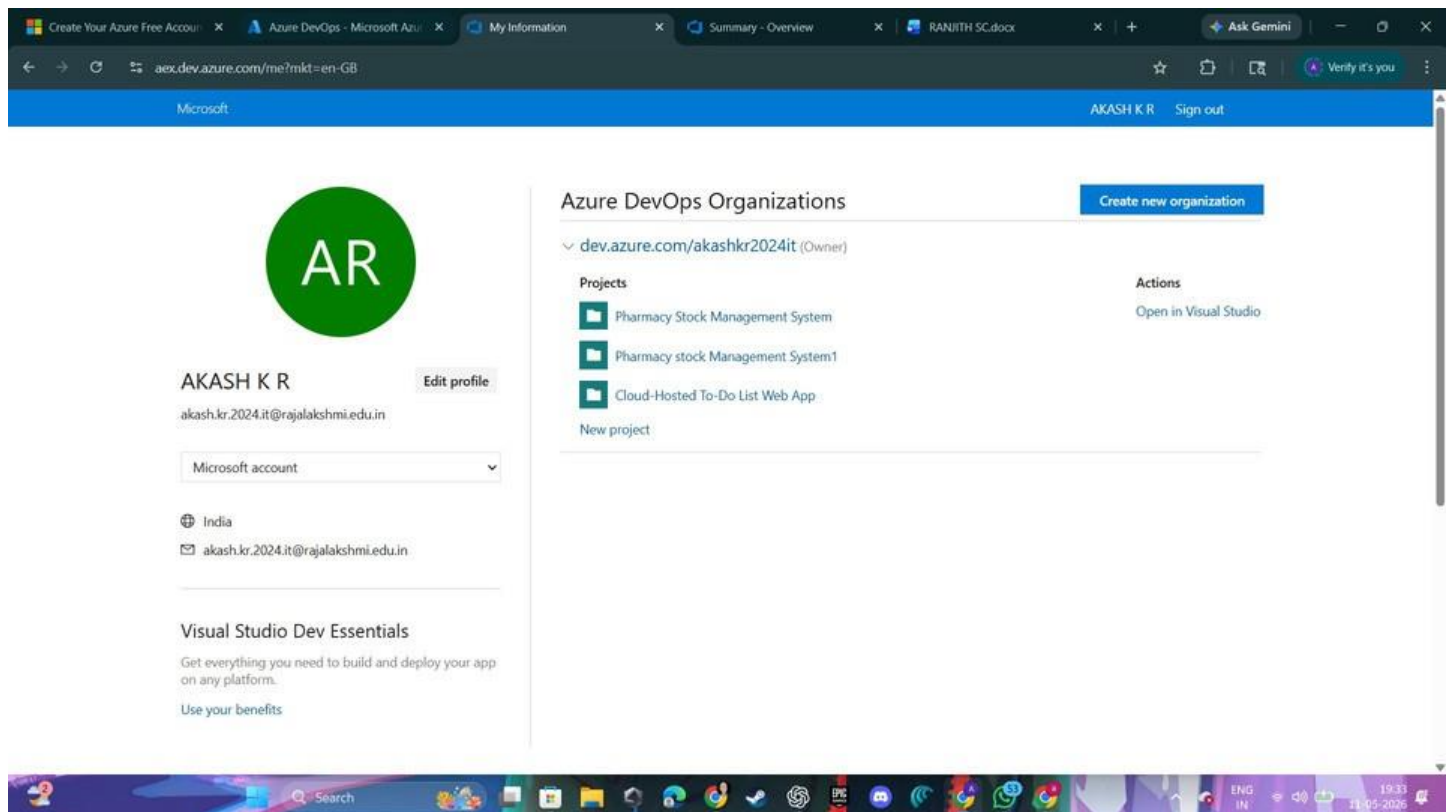
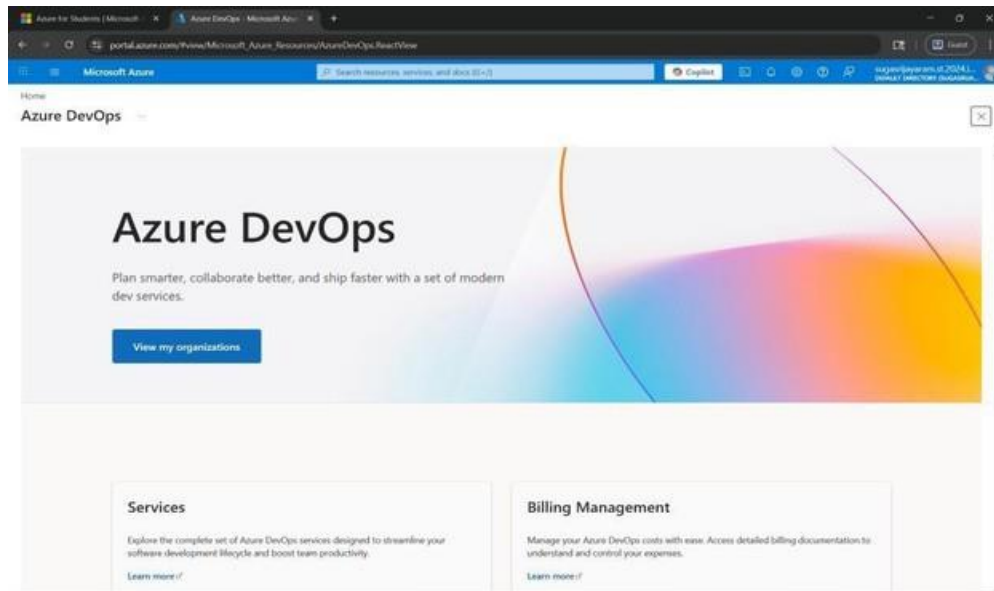
2. Azure home page



3. Open DevOps environment in the Azure platform by typing *Azure DevOps Organizations* in the search bar.



4. Click on the **View My Organization** link and create an organization and you should be taken to the Azure DevOps Organization Home page.



Result:

Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

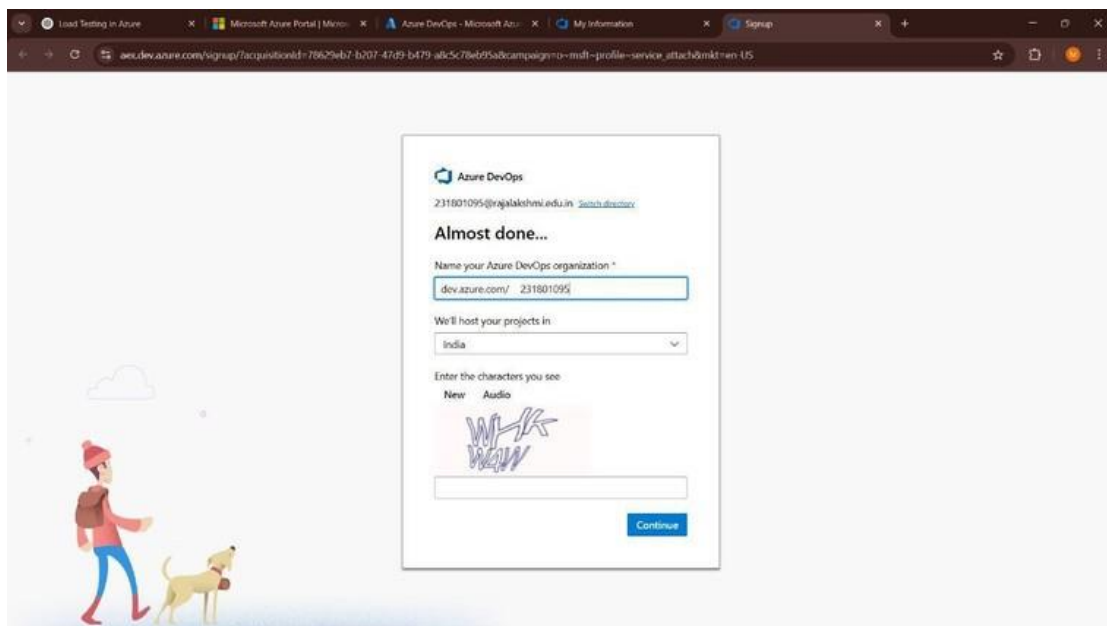
EXP NO: 2 Create Epic, Features, User Stories, Task	AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT
---	---

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

Procedure:

1. Create An Azure Account



2. Create the First Project in Your Organization

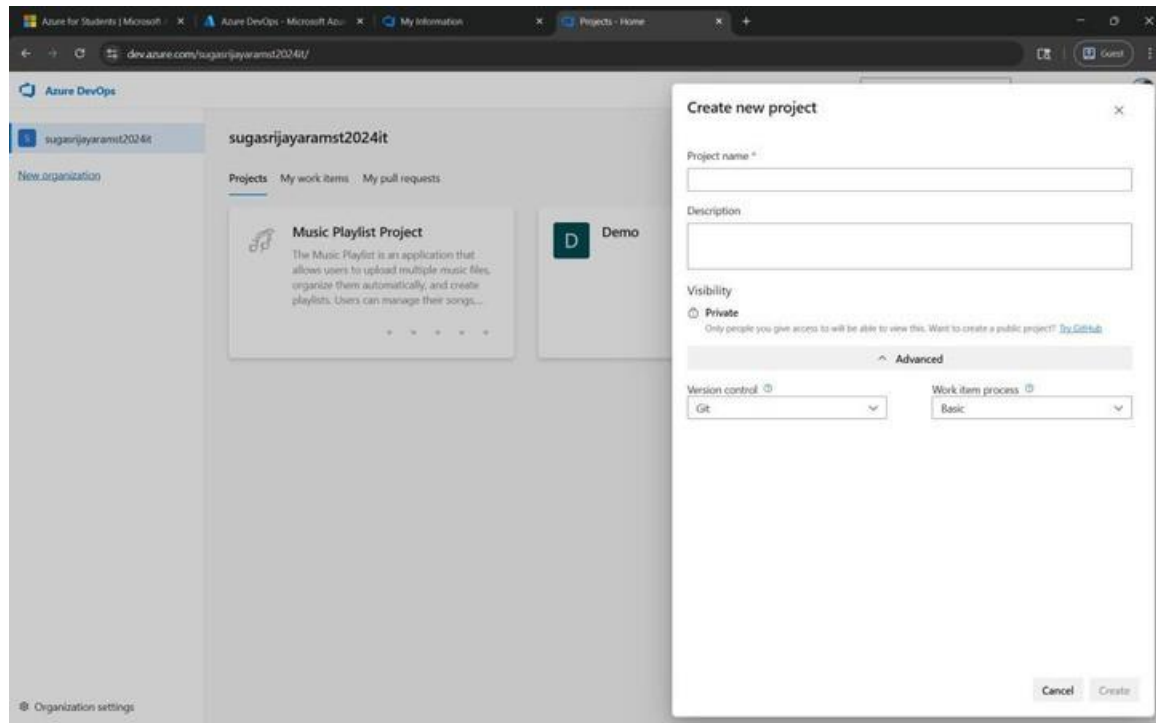
- a. After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.
- b. On the organization's **Home page**, click on the **New Project** button.
- c. Enter the project name, description, and visibility options:

Name: Music Playlist Project.

Description: The Music Playlist is an application that allows users to upload multiple music files, organize them automatically, and create playlists. Users can manage their songs, arrange playlists, and download songs securely after payment. The system also uses external APIs to fetch song metadata and improve music organization.

Visibility: Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).

d. Once you've filled out the details, click **Create** to set up your first project.



3. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.

Create Your Azure Free Account

Azure DevOps - Microsoft Azure

My Information

Summary - Overview

RANJITH SC.docx

Ask Gemini

ax.dev.azure.com/me?mkt=en-GB

MicrosoftAKASH K RSign out

AR

AKASH K R

Edit profile

akash.kr.2024.it@rajalakshmi.edu.in

Microsoft account

India

akash.kr.2024.it@rajalakshmi.edu.in

Visual Studio Dev Essentials

Get everything you need to build and deploy your app on any platform.

Use your benefits

Azure DevOps Organizations

Create new organization

dev.azure.com/akashkr2024it (Owner)

Projects

Pharmacy Stock Management System

Pharmacy stock Management System1

Cloud-Hosted To-Do List Web App

New project

Actions

Open in Visual Studio

4. Project dashboard

The screenshot shows the Azure DevOps Project dashboard for a project named 'Pharmacy stock Management System1'. The left-hand navigation menu includes Overview, Summary, Dashboards, Wiki, Boards, Repos, Pipelines, Test Plans, and Artifacts. The main content area is titled 'About this project' and contains a detailed description of the system. The right-hand sidebar shows 'Project stats' for the last 7 days, with 16 work items and 0 completed items. Below this, the 'Members' section lists three users: AR, AB, and a third user.

Pharmacy stock Management System1

About this project

The Pharmacy Management System is a modern web-based application designed to automate and simplify the various operations performed in a medical store or pharmacy. The main objective of the project is to reduce manual work, improve efficiency, maintain medicine records accurately, and provide a secure and user-friendly platform for pharmacy management. The system is developed using Python and Flask for backend development, while HTML, CSS, JavaScript, and Bootstrap are used to create an attractive and responsive user interface. The database connectivity is implemented using MongoDB or Microsoft Azure Cosmos DB, which stores all medicine details, billing records, sales information, and user credentials securely in the cloud. The project includes several important modules such as user authentication, medicine inventory management, billing system, dashboard analytics, stock monitoring, expiry tracking, and smart medicine search. The login system provides secure access for admins and normal users using role-based authentication, where admins are allowed to add, update, and delete medicine records while normal users have limited access permissions. The inventory management module helps maintain medicine stock details including medicine name, quantity, price, and expiry date, while automatically identifying low-stock medicines and expired products to improve stock control and prevent losses. The billing module generates bills automatically based on the selected medicines and quantities, calculates total prices, updates stock after purchase, and stores sales history for future reference. The dashboard module visually displays important statistics such as total medicines available, low-stock alerts, expired medicines, and total sales, helping the pharmacy management monitor business performance effectively. An AI-based smart medicine search feature is also included in the project, where users can search medicines based on symptoms such as fever, cold, pain, headache, and cough, making the system more intelligent and user-friendly. The project follows concepts of agile project management using epics, features, user stories, sprint planning, and poker planning implemented through Azure DevOps for project tracking and management. The application can be deployed locally or hosted on cloud platforms like Microsoft Azure App Service for online accessibility and scalability. Overall, the Pharmacy Management System is an efficient, secure, scalable, and easy-to-use solution that helps pharmacies manage daily operations digitally, reduce paperwork, improve billing accuracy, enhance inventory management, save time, and increase overall productivity while providing better service to customers.

Project stats Period: Last 7 days

Boards

16 Work items
0 Work items

Members

AR AB

5. To manage user stories:

- From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.
- On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a + button or **Add New Work Item** depending on the view you're in. From the **Add work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.

The screenshot shows the Azure DevOps Boards page for the 'Pharmacy stock Management System1' project. The left-hand navigation menu includes Overview, Summary, Dashboards, Wiki, Boards, Repos, Pipelines, Test Plans, and Artifacts. The main content area is titled 'Work items' and shows a list of work items. The table has columns for ID, Title, Assigned To, State, Area Path, and Tags. The work items are listed in a table with the following data:

ID	Title	Assigned To	State	Area Path	Tags
31	As a user, I want to search medicines using symptoms so that I c...	Unassigned	New	Pharmacy stock Management ...	
29	As a cashier, I want to generate bills automatically so that customer c...	Unassigned	New	Pharmacy stock Management ...	
26	As an admin, I want to log into the system securely so that I can mana...	yaswantkowsnik.c.2024.it...	New	Pharmacy stock Management ...	
30	As an admin, I want sales history reports so that business performance...	Unassigned	New	Pharmacy stock Management ...	
28	As an admin, I want low-stock alerts so that medicines can be restock...	AKASH K R	New	Pharmacy stock Management ...	
27	As an admin, I want to add new medicines so that stock is updated in...	AKASH K R	New	Pharmacy stock Management ...	
18	Billing & Sales Management	ashok r	New	Pharmacy stock Management ...	
16	User Authentication	yaswantkowsnik.c.2024.it...	New	Pharmacy stock Management ...	
17	Inventory Management	AKASH K R	New	Pharmacy stock Management ...	
25	Smart Medicine Search	Unassigned	New	Pharmacy stock Management ...	
24	Sales Tracking	Unassigned	New	Pharmacy stock Management ...	
23	Generate Bills	Unassigned	New	Pharmacy stock Management ...	

USER STORY 3

3 Student Account Registration

No one selected0 CommentsAdd Tag

Save and CloseFollow

Updated by Rangith Bharathwaj G Mar 18

Details

StatgNewAreaCEMS

ReasonMoved to the backlogIterationCEMS

Description

As a student, I want to create an account so that I can access the college event system.

Acceptance Criteria

- Registration form collects name, email, and password
- Email must be unique
- Data is stored successfully in the database
- Confirmation message is displayed

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

switch to Markdown editor

Planning

Story Points3Priority2Risk

Classification

Value areaBusiness

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

Parent

2 User RegistrationUpdated Mar 17 @ New

Result:

Successfully created an Azure DevOps project with user story management and agile workflow setup.

EXP NO: 3

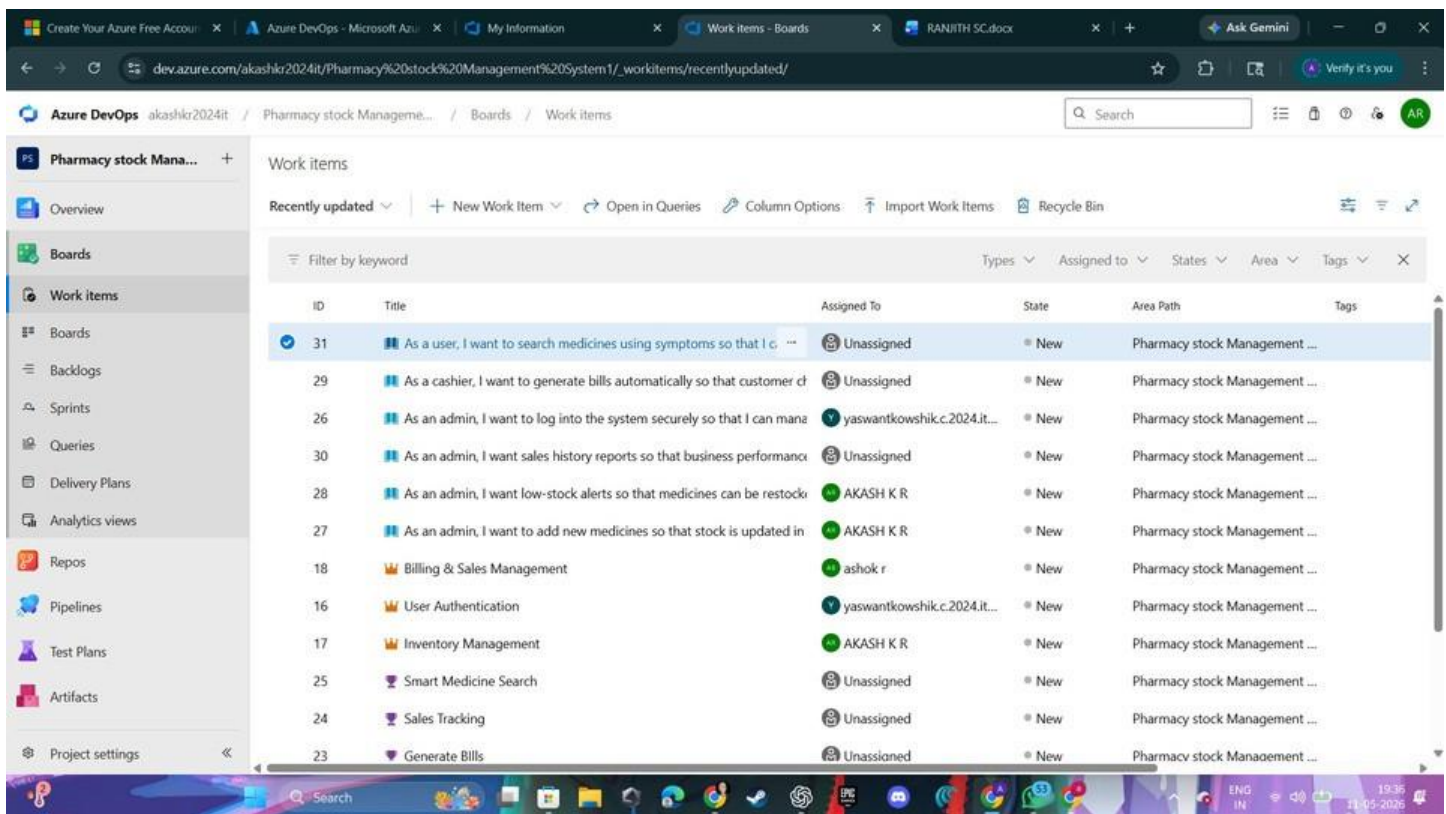
Create Epic, Features, User Stories, Task

SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING

Aim:

To learn about how to create epics, user story, features, backlogs for college event management.

Create Epic, Features, User Stories, Task:



The screenshot shows the Azure DevOps interface for a project named "Pharmacy stock Management". The "Work items" section is active, displaying a list of work items. The interface includes a sidebar with navigation options like Overview, Boards, Work items, Backlogs, Sprints, Queries, Delivery Plans, Analytics views, Repos, Pipelines, Test Plans, Artifacts, and Project settings. The main area shows a table of work items with columns for ID, Title, Assigned To, State, Area Path, and Tags. The work items are filtered by "Recently updated".

ID	Title	Assigned To	State	Area Path	Tags
31	As a user, I want to search medicines using symptoms so that I can...	Unassigned	New	Pharmacy stock Management ...	
29	As a cashier, I want to generate bills automatically so that customer c...	Unassigned	New	Pharmacy stock Management ...	
26	As an admin, I want to log into the system securely so that I can man...	yaswantkowshik.c.2024.it...	New	Pharmacy stock Management ...	
30	As an admin, I want sales history reports so that business performance...	Unassigned	New	Pharmacy stock Management ...	
28	As an admin, I want low-stock alerts so that medicines can be restock...	AKASH K R	New	Pharmacy stock Management ...	
27	As an admin, I want to add new medicines so that stock is updated in...	AKASH K R	New	Pharmacy stock Management ...	
18	Billing & Sales Management	ashok r	New	Pharmacy stock Management ...	
16	User Authentication	yaswantkowshik.c.2024.it...	New	Pharmacy stock Management ...	
17	Inventory Management	AKASH K R	New	Pharmacy stock Management ...	
25	Smart Medicine Search	Unassigned	New	Pharmacy stock Management ...	
24	Sales Tracking	Unassigned	New	Pharmacy stock Management ...	
23	Generate Bills	Unassigned	New	Pharmacy stock Management ...	

Fill in Epics

🔥 EPIC 1

1 User Management

No one selected

0 Comments

Add Tag

State: New

Area: CEMS

Reason: New

Iteration: CEMS

Updated by Ranjith Bharathwaj G: Mar 17

Details

Description

Click to add Description.

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

switch to Markdown editor

Planning

Priority: 2

Risk

Effort

Business Value

Time Criticality

Start Date: Select a date...

Target Date: Select a date...

Classification

Value area: Business

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

Add an existing work item as a parent

Child

🔍 Profile Management
Updated Mar 17 @ New

🔍 User Authentication

Fill in Features

🔥 FEATURE 2

2 User Registration

No one selected

0 Comments

Add Tag

State: New

Area: CEMS

Reason: New

Iteration: CEMS

Updated by Ranjith Bharathwaj G: Mar 17

Details

Description

Click to add Description.

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

switch to Markdown editor

Planning

Priority: 2

Risk

Effort

Business Value

Time Criticality

Start Date: Select a date...

Target Date: Select a date...

Classification

Value area: Business

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

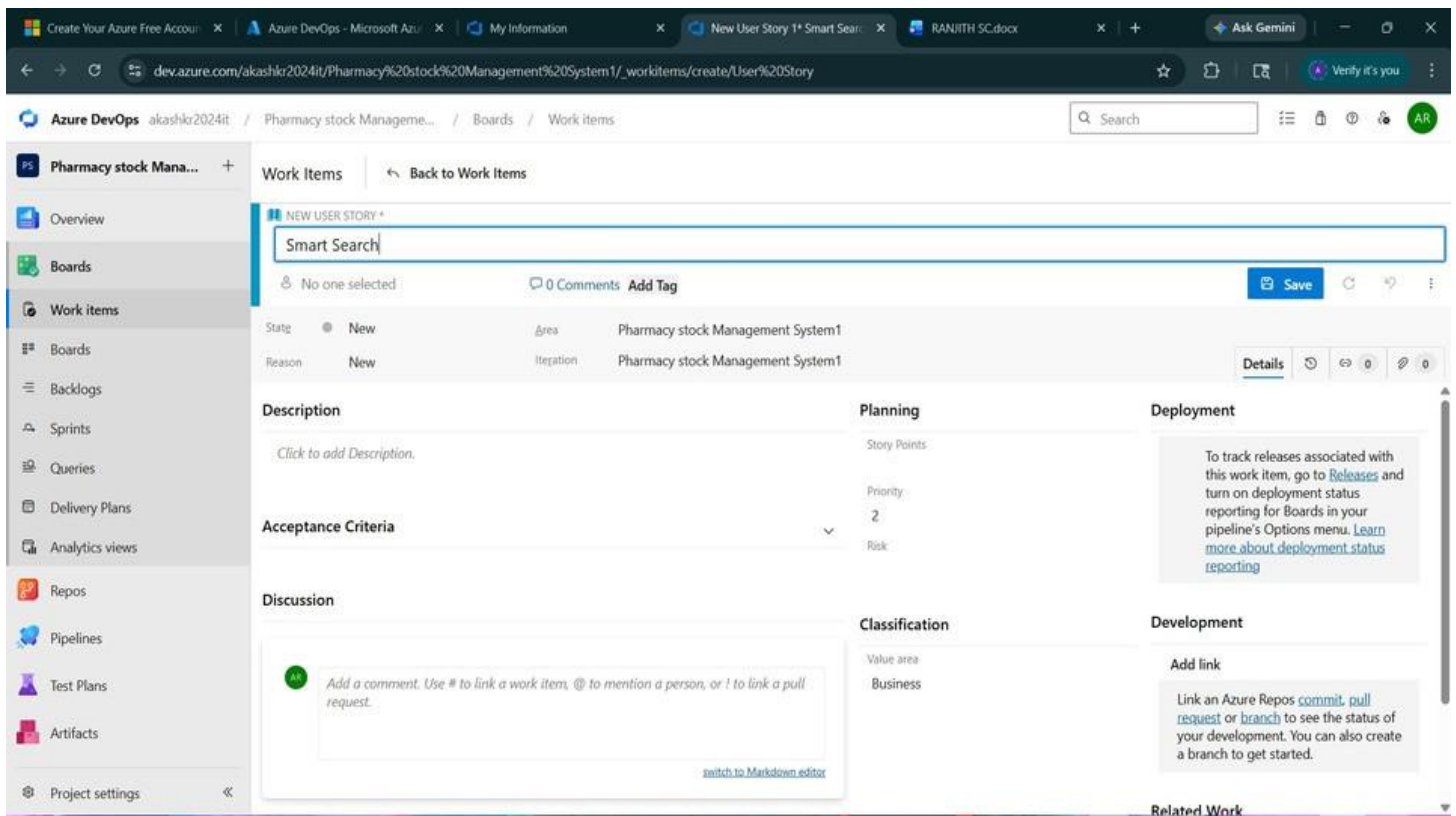
Parent

🔥 1 User Management
Updated Mar 17 @ New

Child

🔍 4 Email Validation
Updated Mar 17 @ New

Fill in User Story Detail



Result:

Thus, the creation of epics, features, user story and task has been created successfully.

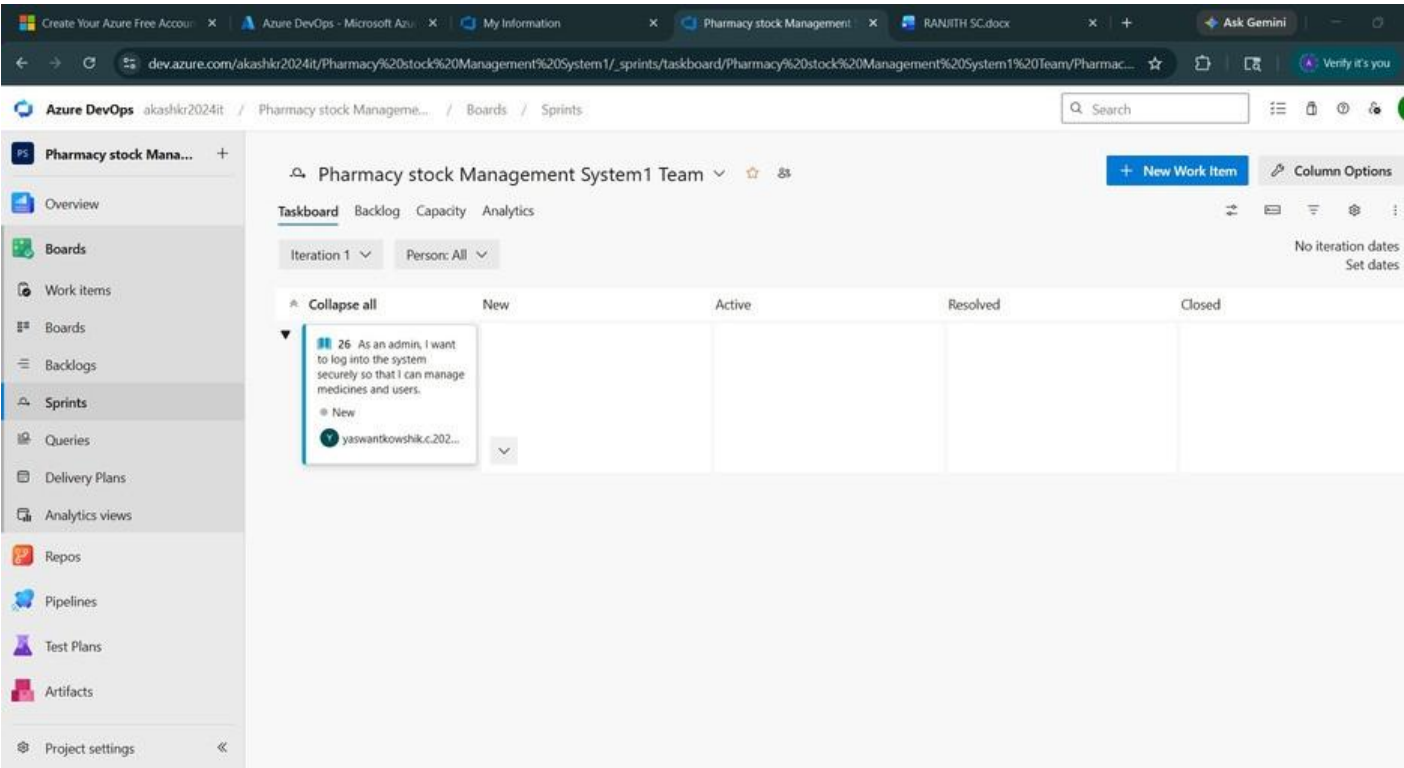
EXPNO: 4
Create Epic,Features, User Stories, Task

SPRINT PLANNING

Aim:
To assign user story to specific sprint for the College event management Project.

Sprint Planning:

Sprint 1



Sprint 2

The screenshot shows the Azure DevOps interface for a project named 'Pharmacy stock Management System1 Team'. The left sidebar contains navigation options: Overview, Boards, Work items, Backlogs, Sprints (selected), Queries, Delivery Plans, Analytics views, Repos, Pipelines, Test Plans, Artifacts, and Project settings. The main area displays the 'Taskboard' view for 'Iteration 2', filtered by 'Person: All'. The taskboard is organized into columns: New, Active, Resolved, and Closed. Two work items are visible in the 'New' column:

- Item 27:** As an admin, I want to add new medicines so that stock is updated in the database. (New, assigned to AKASH K R)
- Item 28:** As an admin, I want low-stock alerts so that medicines can be restocked before shortage occurs. (New, assigned to AKASH K R)

Each item has a dropdown arrow for more details. The top right of the interface includes a search bar, a 'New Work Item' button, and 'Column Options'. The bottom right corner indicates 'No iteration dates' and a 'Set dates' link.

Sprint 3

Create Your Azure Free Account

Azure DevOps - Microsoft Azure

My Information

Pharmacy stock Management

RANJITH SC.docx

Ask Gemini

dev.azure.com/akashkr2024it/Pharmacy%20stock%20Management%20System1/_sprints/taskboard/Pharmacy%20stock%20Management%20System1%20Team/Pharmac...

Verify it's you

Azure DevOps

akashkr2024it / Pharmacy stock Manage...

Boards / Sprints

Search

AR

Pharmacy stock Mana...

Overview

Boards

Work items

Boards

Backlogs

Sprints

Queries

Delivery Plans

Analytics views

Repos

Pipelines

Test Plans

Artifacts

Project settings

Pharmacy stock Management System1 Team

New Work Item

Column Options

Taskboard

Backlog

Capacity

Analytics

Iteration 3

Person: All

No iteration dates

Set dates

Collapse all

New

Active

Resolved

Closed

29 As a cashier, I want to generate bills automatically so that customer checkout becomes faster.
New
Unassigned

30 As an admin, I want sales history reports so that business performance can be analyzed.
New
Unassigned

31 As a user, I want to search medicines using symptoms so that I can find suitable medicines quickly.
New

Sprint 4

Create Your Azure Free Account

Azure DevOps - Microsoft Azure

My Information

Pharmacy stock Management

RANJITH SC.docx

Ask Gemini

dev.azure.com/akashkr2024it/Pharmacy%20stock%20Management%20System1/_sprints/taskboard/Pharmacy%20stock%20Management%20System1%20Team/Pharmac...

Verify it's you

Azure DevOps

akashkr2024it / Pharmacy stock Manage...

Boards / Sprints

Search

AR

Pharmacy stock Mana...

Overview

Boards

Work items

Boards

Backlogs

Sprints

Queries

Delivery Plans

Analytics views

Repos

Pipelines

Test Plans

Artifacts

Project settings

Pharmacy stock Management System1 Team

New Work Item

Column Options

Taskboard

Backlog

Capacity

Analytics

Iteration 3

Person: All

No iteration dates

Set dates

Collapse all

New

Active

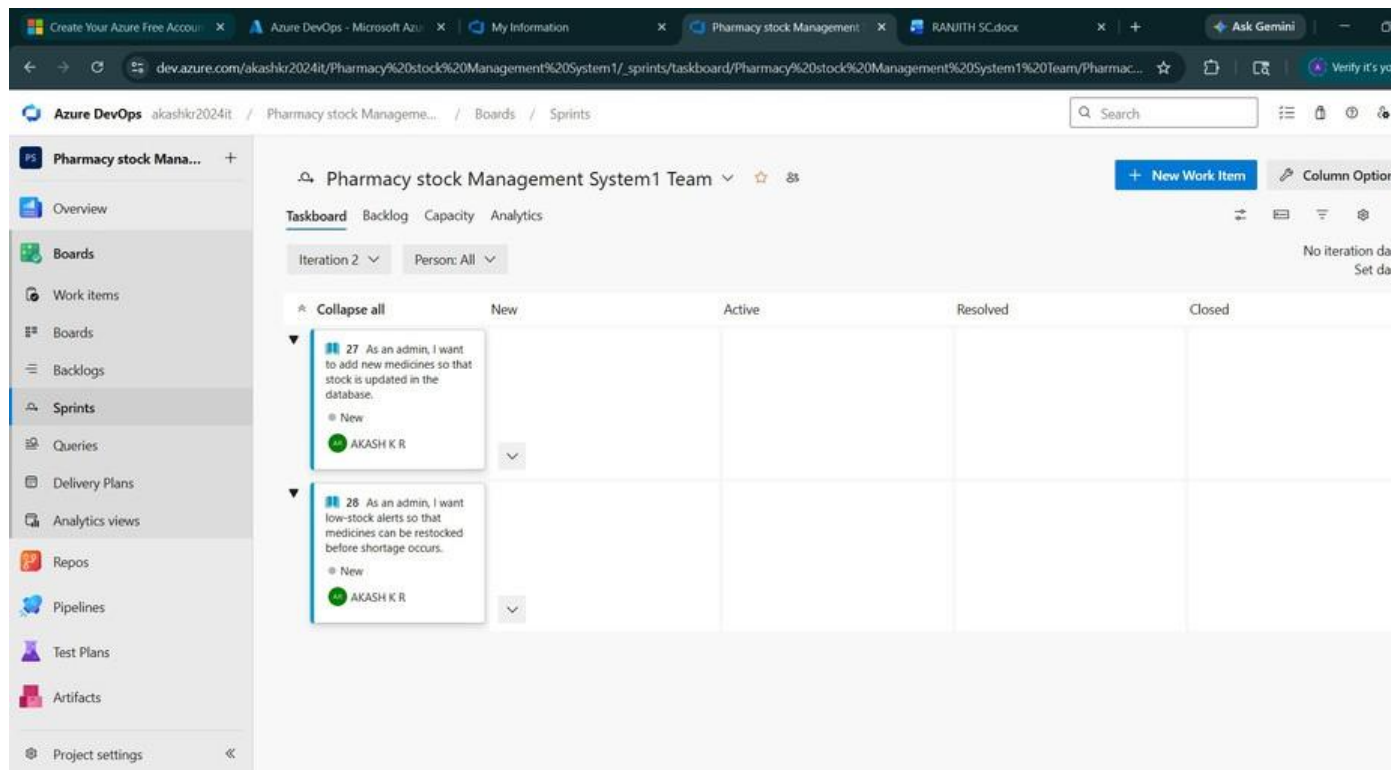
Resolved

Closed

29 As a cashier, I want to generate bills automatically so that customer checkout becomes faster.
New
Unassigned

30 As an admin, I want sales history reports so that business performance can be analyzed.
New
Unassigned

31 As a user, I want to search medicines using symptoms so that I can find suitable medicines quickly.
New



Result:

The Sprints are created for the College event management.

EXPNO: 5 Create Epic, Features, User Stories, Task	POKER ESTIMATION
--	-------------------------

Aim:

Create Poker Estimation for the user stories for Music Playlist Project.

Planning Poker Estimation:

User Story ID: 20

Title: User Registration and Authentication

Description:

As a user, I want to register and log in securely so that I can access my personal music playlists and downloads.

Story Point Estimation: 8 Points

Team Poker Votes:

Team	Member	Vote (Story Points)
Team	Member 1	5
Team	Member 2	8
Team	Member 3	8
Team	Member 4	8

Final Consensus: 8 Points

Reason for Estimation:

- Includes multiple features: registration, login, session, logout
- Requires frontend + backend + database integration
- Security implementation (password hashing, authentication)
- Moderate complexity with high risk

Risks:

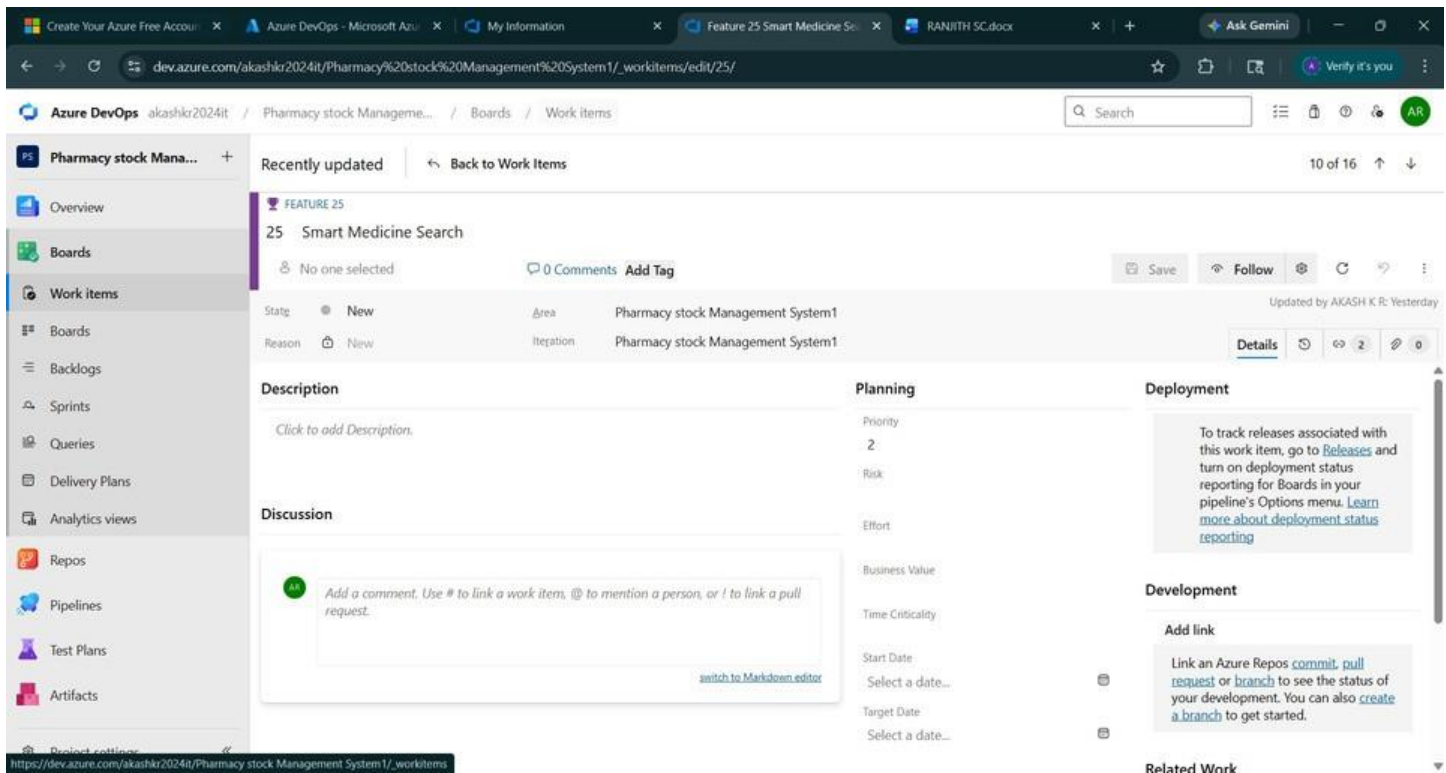
- Security vulnerabilities
- Session management issues
- Integration challenges

Acceptance Criteria Covered:

- User registration with email & password
- Login with valid credentials
- Error message for invalid login
- Session maintained after login
- Logout functionality

Conclusion:

Final agreed estimation is **8 story points** based on complexity, effort, and risk.



Result:

The Estimation/Story Points is created for the project using Poker Estimation.

EXPNO: 6 Create Epic, Features, User Stories, Task	DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE
--	---

Aim:

To Design a Class Diagram and Sequence Diagram for the College Event Management.

Class Diagram:

Classes:

- Student
- Admin
- Event
- Registration
- Payment
- Certificate
- Notification
- Coordinator

Key Attributes:

Student:

StudentId, Name, Email, Department

Event:

EventId, Title, Date, Venue, Category, Fee

Registration:

RegistrationId,StudentId,E

ventId,PaymentStatus

Payment:

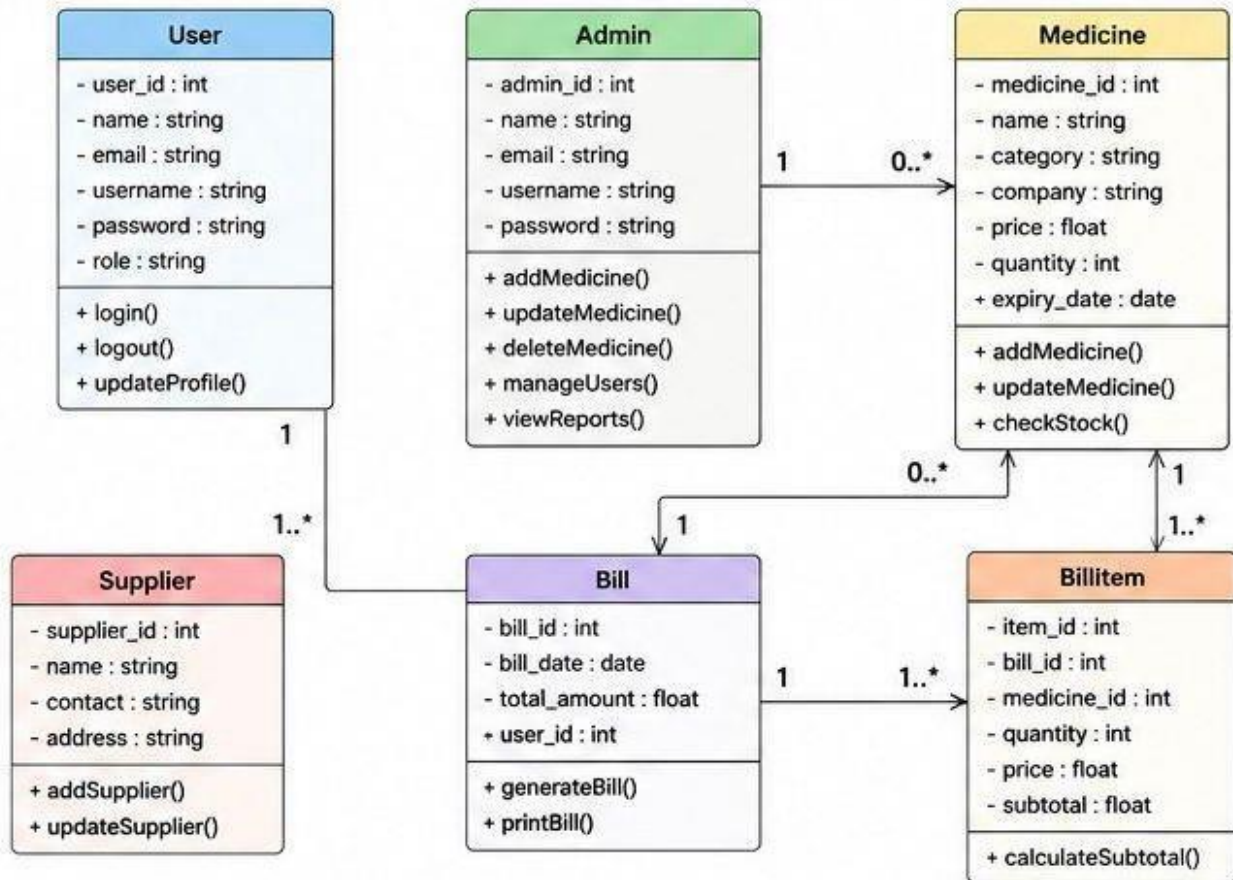
PaymentId,Amount,Tran

sactionStatus

Relationships:

- Student → registers for → Event
- Activity → conducted at → Venue
- Student → participates in → Competition
- Faculty Coordinator → manages → Event
- Organizer → schedules → Event
- Event → has → Participants
- Student → makes → Payment
- Student → receives → Ticket/Pass
- Student → submits → Feedback
- Student → receives → Notification
- Sponsor → sponsors → Event
- Event → awards → Certificate/Prize
- Department → organizes → Event
- Venue → allocated to → Event
- Student → joins → Team
- Team → participates in → Event
- Admin → approves → Event Request
- Event → publishes → Results
- Offer/Discount → applies to → Event Registration

CLASS DIAGRAM – PHARMACY MANAGEMENT SYSTEM



Sequence Diagram:

Actors:

User → Frontend → Backend → Database → Auth Service

Steps:

1. User enters email and password
2. Frontend validates input (email format, password)
3. Frontend sends login request to backend
4. Backend validates request data
5. Backend checks user in database

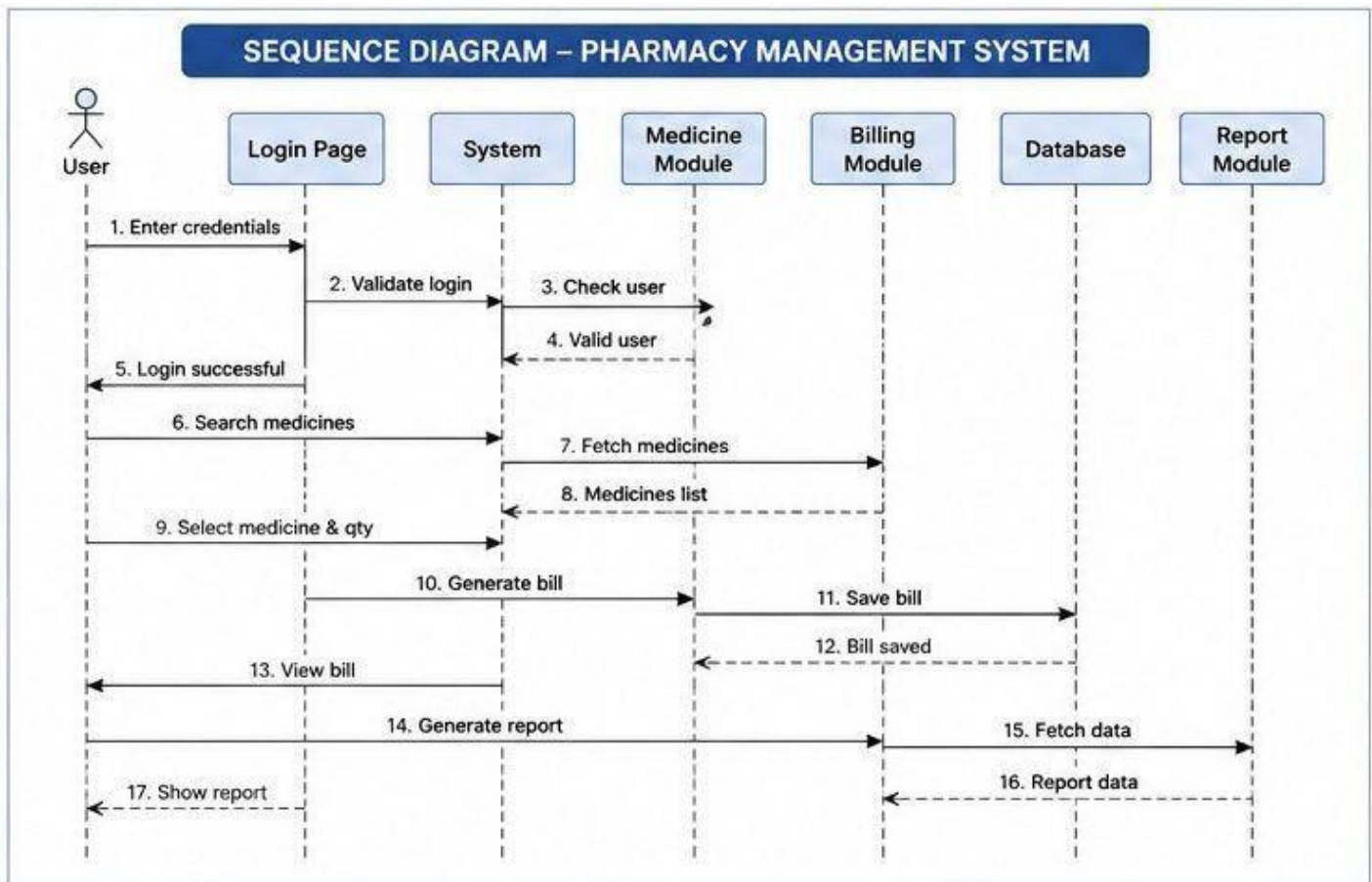
Condition: User Not Found

- Backend returns error: *Invalid credentials*

Condition: User Found

6. Backend verifies password
7. If password incorrect → return error
8. If password correct → proceed
9. Backend verifies user with auth service
10. Backend generates JWT token
11. Backend sets session (token expiry)
12. Backend sends success response

13. Frontend stores token
14. Frontend redirects to dashboard
15. User login successful



Result: The Class Diagram and Sequence Diagram is designed Successfully for the College Event Managemnt.

EXPNO: 7

Create Epic, Features, User Stories, Task

DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE

Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

Architectural Diagram:

Layers:

1. Client Layer

- Browser
- React App
- Zustand (State Management)

2. API Communication

- Axios (HTTP Requests)

3. Backend Layer

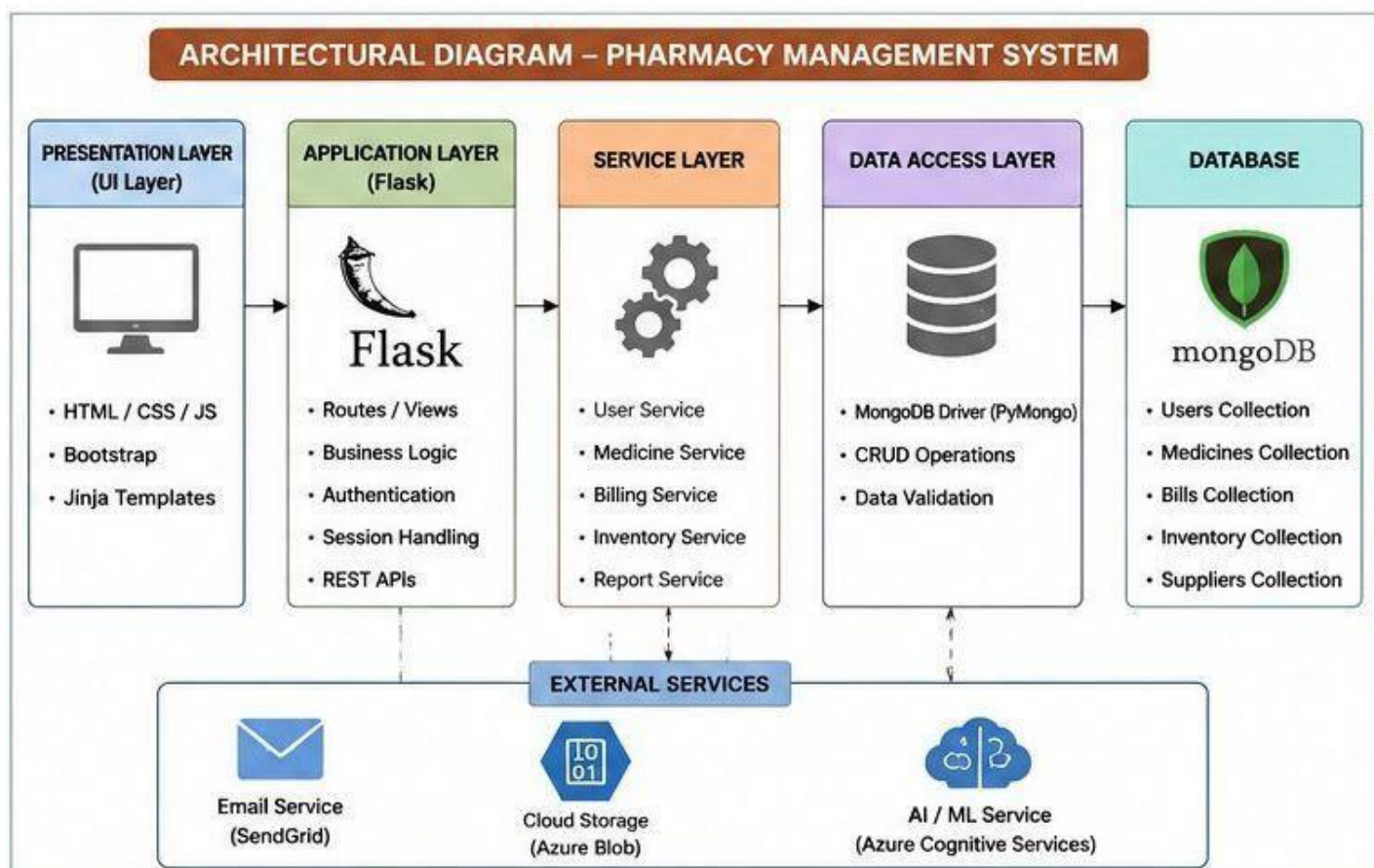
- Express Server
- Routes
- Controllers

4. Services

- Clerk Authentication
- File Upload(Cloudinary)

5. Database Layer

- MongooseModels
- MongoDB



ER Diagram:

Entities & Attributes:

Student

- StudentId
- Name Email
- Department
-

Event:

- EventId
- Title
- Venue

- Date
- Fee

Registration

- registration
- amount
- status

Payment

- paymentId
- amount
- status

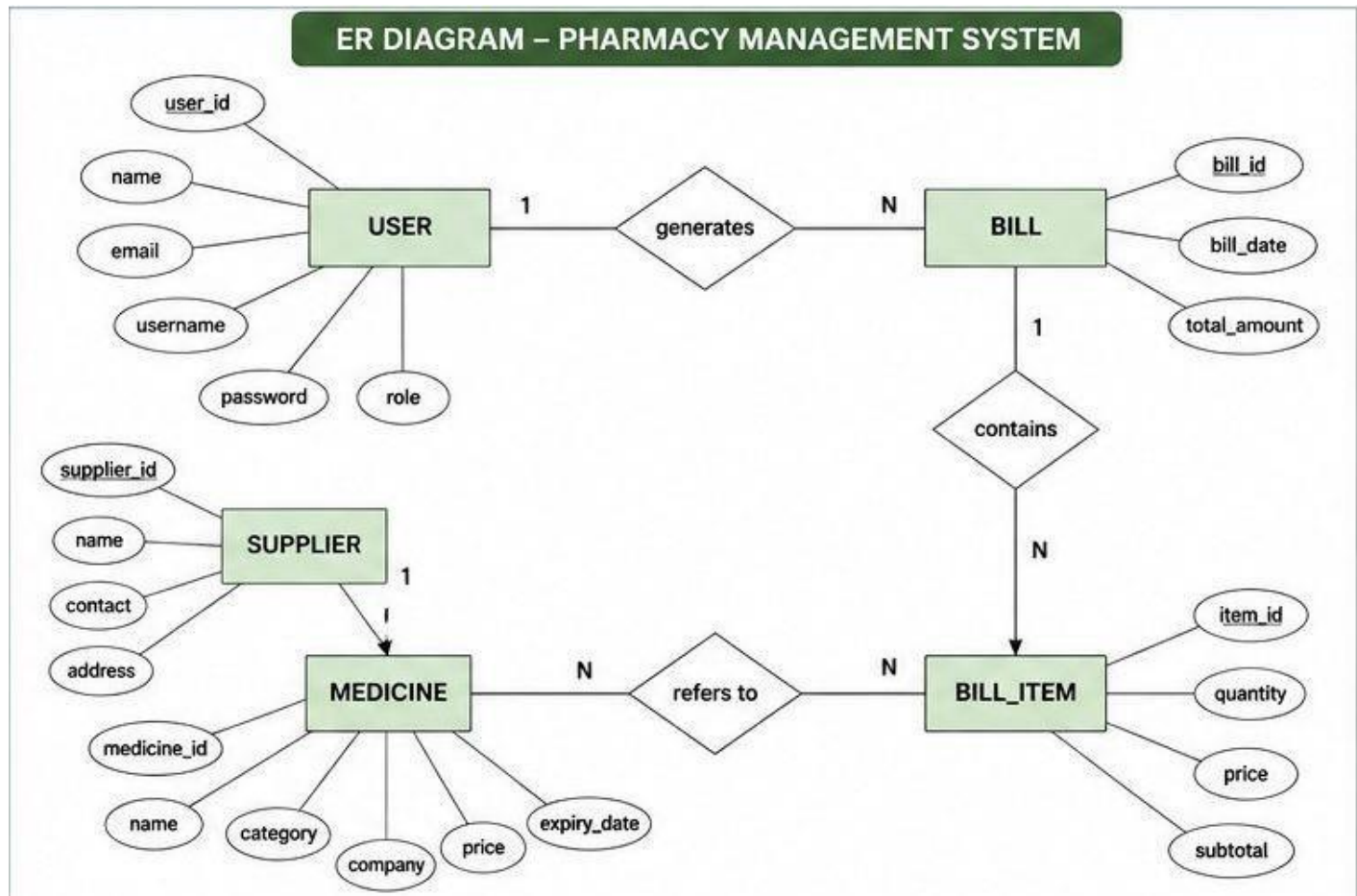
certificate

- certificateId

- issueDate
-

Relationships

- Student registers for Event (M : N)
- Event includes Activity (1 : M)
- Activity conducted at Venue (M : 1)
- Student participates in Competition (M : N)
- Faculty Coordinator manages Event (1 : M)
- Organizer schedules Event (1 : M)
- Event has Participants (1 : M)
- Student makes Payment (1 : M)
- Student receives Ticket / Pass (1 : 1)
- Student submits Feedback (1 : M)
- Student receives Notification (1 : M)
- Sponsor sponsors Event (M : N)
- Event awards Certificate / Prize (1 : M)
- Department organizes Event (1 : M)
- Venue allocated to Event (1 : M)
- Student joins Team (M : N)
- Team participates in Event (M : N)
- Admin approves Event Request (1 : M)
- Event publishes Results (1 : 1)
- Offer / Discount applies to Event Registration (1 : M)



Result: The Architecture Diagram and ER Diagram is designed Successfully for the College Event Management.

EXP NO: 8	TESTING – TEST PLANS AND TEST CASES
Create Epic, Features, User Stories, Task	

Aim:

Test Plans and Test Case and write two test cases for at least two user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Planning and Test Case:

Test Case Design Procedure

1. Understand Core Features of the Application

- o User Signup & Login
- o Viewing and Managing Playlists
- o Fetching Real-time Metadata
- o Editing playlists (rename, reorder, record)
- o Creating smart audio playlists based on categories (mood, genre, artist, etc.)

2. Define User Interactions

- o Each test case simulates a real user behaviour (e.g., logging in, renaming a playlist, adding a song).

3. Design Happy Path Test Cases

- o Focused on validating that all features function as expected under normal conditions.
- o Example: User logs in successfully, adds item to playlist, or creates a category-based playlist.

4. Design Error Path Test Cases

- o Simulate negative or unexpected scenarios to test robustness and error handling.
- o Example: Login fails with invalid credentials, save fails when offline, no recommendations found.

5. Break Down Steps and Expected Results

- o Each test case contains step-by-step actions and a corresponding expected outcome.
- o Ensures clarity for both testers and automation scripts.

6. Use Clear Naming and IDs

- o Test cases are named clearly (e.g., TC01 – Successful signup, TC06 – Add and Remove Songs from Playlist).
- o Helps in quick identification and linking to user stories or features.

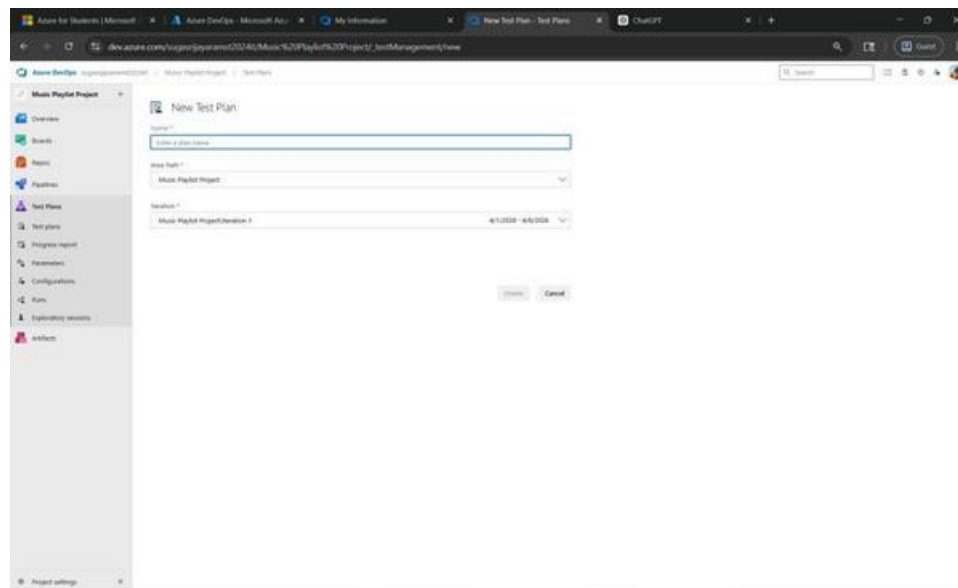
7. Separate Test Suites

- o Grouped test cases based on functionality (e.g., Login, Playlist Editing, Recommendation System).
- o Improves organization and test execution flow in Azure DevOps.

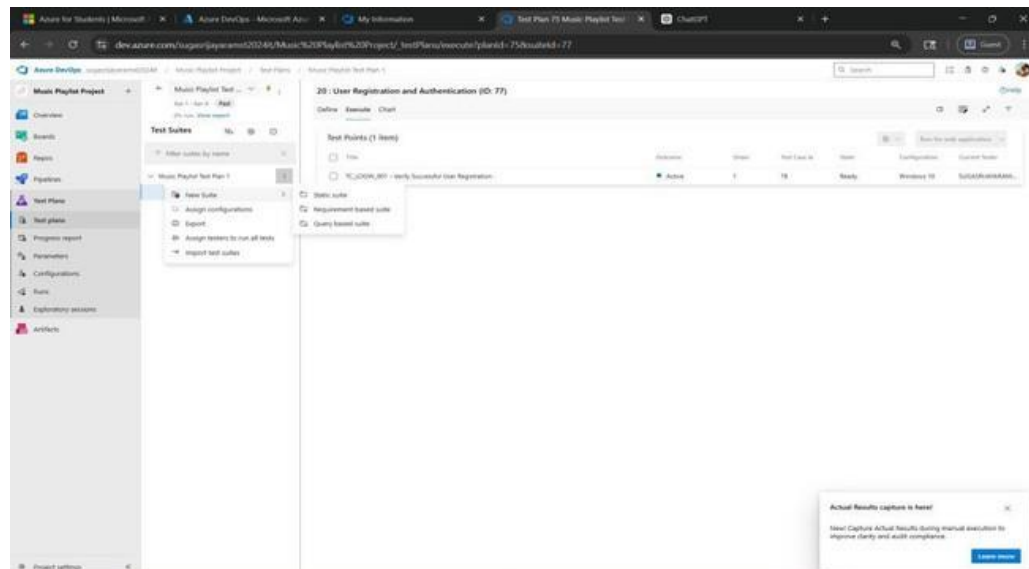
8. Prioritize and Review

- o Critical useractions are marked high-priority.
- o Reviewed for completeness and traceability against feature requirements.

1. New test plan



2. Test suite



3. Test case

Give two test cases for at least two user stories showcasing the happy path and error scenarios in azure DevOps platform.

Music Playlist Project – Test Plans

USER STORIES

- As a user, I want to register and log in securely so that I can access my personal music playlists and downloads. (ID: 20).
- As a user, I want to create and manage playlists so that I can organize my favourite songs easily (ID: 23).

Test Suites

Test Suit: TS01 - User Registration and Authentication (ID: 20)

1. TC01 – User Registration Successfully w/o Error

- **Action:**
 - Go to the Sign-Up page
 - Enter valid name, email, and password
 - Click "Sign Up"
- **Expected Results:**
 - Sign-Up form is displayed
 - Fields accept values without error
 - Account is created successfully
 - User is redirected to dashboard
- **Type: Happy Path**

2. TC02 – User Registration Failed with Invalid User Details

- **Action:**
 - Go to the Sign-Up page
 - Enter invalid details (empty fields / wrong email format / weak password)
 - Click "Sign Up"
- **Expected Results:**
 - Validation errors are displayed
 - Account is not created
- **Type: Error Path**

3. TC03 – User Login Successfully with Valid Credentials

- **Action:**
 - Go to Login page
 - Enter valid email and password
 - Click "Login"
- **Expected Results:**
 - Login form is displayed
 - Credentials are accepted
 - User is redirected to dashboard
- **Type: Happy Path**

4. TC04 – Login with Invalid Credentials

- **Action:**
 - Go to Login page
 - Enter invalid email or password
 - Click "Login"
- **Expected Results:**
 - Input is accepted
 - Error message "Invalid username or password" is shown
- **Type: Error Path**

Test Suit: TS02 - Playlist Creation and Management (ID: 23)

1. TC05 – Create Playlist Successfully

- **Action:**
 - Login with valid credentials
 - Navigate to "My Playlists"
 - Click "Create Playlist"
 - Enter playlist name
 - Click "Save/Create"
- **Expected Results:**
 - Playlist is created successfully
 - Playlist appears in list
- **Type: Happy Path**

2. TC06 – Add and Remove Songs from Playlist

- o **Action:**
 - o Open an existing playlist
 - o Click "Add Song" and select a song
 - o Click "Remove" on a song
- o **Expected Results:**
 - o Song is added to playlist
 - o Song is removed from playlist
- o **Type:** Happy Path

3. TC08 – Delete Playlist and Save Changes

- o **Action:**
 - o Navigate to "My Playlists"
 - o Click "Delete" on a playlist
 - o Confirm deletion
- o **Expected Results:**
 - o Playlist is deleted successfully
 - o Playlist is removed from list
 - o Changes are saved

Type: Happy Path

Test Cases

The screenshot displays the Azure DevOps Test Plans interface. The left sidebar shows the project structure with 'Music Playlist Project' selected. The main area shows the '20 : User Registration and Authentication (ID: 77)' test plan. The test plan is in a 'Pass' state, indicating 100% run and 100% passed. The 'Test Suites' section shows a list of test suites, including '23 : Playlist Creation and Ma...' and '20 : User Registration and A...'. The 'Test Cases (4 items)' table lists the following test cases:

Title	Order	Test Case Id	Assigned To	State
TC01 – User Registration Successfully w/o Error	1	78	SUGASRIJAYAR...	Close
TC02 – User Registration Failed with Invalid User Details	2	79	SUGASRIJAYAR...	Close
TC03 – User Login Successfully with Valid Credentials	3	80	Sridhar C	Close
TC04-Login with invalid credentials	4	81	Sridhar C	Close

TEST CASE 78

78 TC01 – User Registration Successfully w/o Error

SUGASRIJAYARAM S T

0 Comments Add Tag

Save and Close Follow

Updated by SUGASRIJAYARAM S T: 1h ago

State: Closed Area: Music Playlist Project Reason: Obsolete Iteration: Music Playlist Project/Iteration 1

Steps

Steps	Action	Expected result	Attachments
1.	Navigate to web page	Login page loads	
2.	Click "Get Started"	Registration form appears	
3.	Enter first name and last name	Name fields accept input	
4.	Enter Username	Username field accepts input	
5.	Enter valid email address	Email field accepts input	
6.	Enter password	Password field accepts input	
7.	Click "Continue" button	Account is created	
8.	Check email and enter verification code	Verification is successful	
9.	Click "Continue"	User is redirected to Home page	

Click or type here to add a step

Parameter values

Recent test results

- Passed by SUGASRIJAYARAM S T with Windows 10 Completed 12:07 PM, 4/28/2026
- Failed by SUGASRIJAYARAM S T with Windows 10 Completed 12:06 PM, 4/28/2026
- Passed by SUGASRIJAYARAM S T with Windows 10 Completed 3:20 AM, 4/28/2026

[View all test result history](#)

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

20 : User Registration and Authentication

Define Execute Chart

Test Suites

Filter suites by name

Music Playlist Test Plan 1

23: Playlist Creation and Management

20: User Registration and Authentication

Test Points (4 items)

Title
☒ TC01 – User Registration Successfully w/o Error
☐ TC02 – User Registration Failed with Invalid Email
☐ TC03 – User Login Successfully with Valid Credentials
☐ TC04 – Login with invalid credentials

Test point ID 2 execution history

Test Case: 78 TC01 – User Registration Successfully w/o Error

Test Suite: 77 20 : User Registration and Authentication

Current Tester: SUGASRIJAYARAM S T

Configuration: Windows 10

Outcome	Test run	Run by	Time completed
Passed	9	SUGASRIJAYARAM S...	Yesterday
Failed	8	SUGASRIJAYARAM S...	Yesterday
Passed	5	sirram S	Yesterday

Currently showing a test point execution history. View all history for a full test case execution history. [View all history](#)

Azure for Students x Azure DevOps x My Information x User Story 23 Pl... x Test Plan 75 Mus... x ChatGPT x Golden Span... x New tab x

dev.azure.com/sugarsriyarams2024it/Music%20Playlist%20Project/_testPlans/define?planId=75&outletId=77

TEST CASE 81

81 TC04-Login with invalid credentials

Sridhar C 0 Comments Add Tag Save and Close Follow

State: Closed Area: Music Playlist Project Reason: Obsolete Iteration: Music Playlist Project/Iteration 1 Updated by SUGASRIYARAM S T: 43m ago

Steps Summary Associated Automation 2 0

Steps

Steps	Action	Expected result	Attachments
1.	Open login page	Login page is displayed	
2.	Enter invalid username/email	Input is accepted	
3.	Enter invalid password	Input is accepted	
4.	Click "Login"	System validates credentials	
5.	Login attempt fails	Error message displayed: "Invalid credentials"	

Click or type here to add a step

Recent test results

Passed by with Windows 10 Completed 12:28 PM, 4/26/2026

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

Parent

20 User Registration and Authentication Updated Yesterday Resolved

Tests

20 User Registration and Authentication

Azure for Students x Azure DevOps x My Information x User Story 23 Pl... x Test Plan 75 Mus... x ChatGPT x Golden Span... x New tab x

dev.azure.com/sugarsriyarams2024it/Music%20Playlist%20Project/_testPlans/define?planId=75&outletId=83

TEST CASE 85

85 TC06 - Add and Remove Songs from Playlist

SUGASRIYARAM S T 0 Comments Add Tag Save and Close Follow

State: Closed Area: Music Playlist Project Reason: Obsolete Iteration: Music Playlist Project/Iteration 1 Updated by SUGASRIYARAM S T: 43m ago

Steps Summary Associated Automation 2 0

Steps

Steps	Action	Expected result	Attachments
1.	Open an existing playlist	Playlist details are displayed	
2.	Click "Add Song"	Song selection options appear	
3.	Select a song	Song is added to the playlist	
4.	Verify playlist	Added song is visible in the list	
5.	Click "Remove/Delete" on a song	Song is removed	
6.	Verify playlist	Removed song no longer appears	

Click or type here to add a step

Recent test results

Passed by with Windows 10 Completed 4:36 PM, 4/29/2026

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

Parent

23 Playlist Creation and Management Updated 54m ago Resolved

Tests

23 Playlist Creation and Management

4. Running the test cases

23: Playlist Creation and Management (ID: 83)

Define Execute Chart

Test Points (3 items)

Title	Outcome	Order	Test Case Id	Status
TC05 - Create Playlist Successfully	Passed	1	84	OK
TC06 - Add and Remove Songs from Playlist	Passed	2	85	OK
TC08 - Delete Playlist and Save Changes	Passed	3	86	OK

Context menu for TC06:

- View execution history
- ✓ Mark Outcome
- Run
- Reset test to active
- Edit test case
- Assign tester
- View test result

Run for web application

Run for desktop application

Run with options

85: TC06 - Add and Remove Songs from Playlist

- Open an existing playlist
EXPECTED RESULT: Playlist details are displayed
ACTUAL RESULT: [Pass]
- Click "Add Song"
EXPECTED RESULT: Song selection options appear
ACTUAL RESULT: [Pass]
- Select a song
EXPECTED RESULT: Song is added to the playlist
ACTUAL RESULT: [Pass]
- Verify playlist
EXPECTED RESULT: Added song is visible in the list
ACTUAL RESULT: [Pass]
- Click "Remove/Delete" on a song
EXPECTED RESULT: Song is removed
ACTUAL RESULT: [Pass]
- Verify playlist
EXPECTED RESULT: Removed song no longer appears
ACTUAL RESULT: [Pass]

5. Creating the bug

The screenshot shows the Test Plans interface in Google Chrome. The main window displays a test case titled "85*: TC06 - Add and Remove Songs from Playlist". The test case steps are as follows:

1. Open an existing playlist
EXPECTED RESULT: Playlist details are displayed
2. Click "Add Song"
EXPECTED RESULT: Song selection options appear
COMMENT: Add notes or context
3. Select a song
EXPECTED RESULT: Song is added to the playlist
COMMENT: Add notes or context
4. Verify playlist
EXPECTED RESULT: Added song is visible in the list
COMMENT: Add notes or context
5. Click "Remove/Delete" on a song
EXPECTED RESULT: Song is removed
COMMENT: Add notes or context
6. Verify playlist
EXPECTED RESULT: Removed song no longer appears
COMMENT: Add notes or context

On the right side, a test run is visible with the following data:

Outcome	Order	Test Case Id	Status
Passed	1	84	OK
Passed	2	85	OK
Passed	3	86	OK

The screenshot shows the "Bug01- TC06 Add and Remove Songs from playlist" page. The page includes a "NEW BUG" header, a "Save and Close" button, and a "Details" tab. The "Repro Steps" section shows the following steps:

Step no.	Result	Title
1.	Passed	Open an existing playlist Expected Result: Playlist details are displayed
2.	Failed	Click "Add Song" Expected Result: Song selection options appear
3.	Failed	Select a song Expected Result: Song is added to the playlist

The "System Info" section includes a link to "Click to add System Info". The "Discussion" section is at the bottom. The "Planning" section on the right includes a "Resolved Reason" field, a "Story Points" field, a "Priority" field (set to 2), a "Severity" field (set to 3 - Medium), and an "Activity" field. The "Effort (Hours)" section includes an "Original Estimate" field, a "Remaining" field, and a "Completed" field. The "Deployment" section includes a link to "Add link" and a "Related Work" section. The "Development" section includes a link to "Add link" and a "System Info" section.

6. Test case results

The screenshot shows the Azure DevOps interface for a project named 'Music Playlist Project'. The 'Test Plans' section is active, displaying a test plan titled '20: User Registration and Authentication'. The test plan is in a 'Past' state, with a progress bar indicating 85% completion (100% passed). A modal window titled 'Test point ID 2 execution history' is open, showing a table of test results for test point ID 2.

Outcome	Test run	Run by	Time completed
Passed	21	SUGASRIJAYARAM S...	19m ago
Passed	9	SUGASRIJAYARAM S...	Yesterday
Failed	8	SUGASRIJAYARAM S...	Yesterday
Passed	5	sriam S	Yesterday

Currently showing a test point execution history. View all history for a full test case execution history. [View all history](#)

7. Bug report

- Assigning bug to the developer and changing state to Active

The screenshot shows the Azure DevOps interface for a bug report titled 'Bug01 - TC06 Add and Remove Songs from Playlist'. The bug is in the 'Active' state, with a priority of 2 and a severity of 3 - Medium. The bug is assigned to 'SUGASRIJAYARAM S T' and was updated 5m ago. The bug report includes a list of steps to reproduce the issue, with the first step being 'Click "Add Song"'. The bug is also linked to a test plan '23: Playlist Creation and Management' and a test case 'TC06 - Add and Remove Songs from Playlist'.

Steps to Reproduce:

1. Click "Add Song"
Expected Result: Song selection options appear
Actual Result: Failed
2. Select a song
Expected Result: Song is added to the playlist
Actual Result: Failed

System Info:

- Found in Build
- Integrated in Build

87 Bug01 - TC06 Add and Remove Songs from Playlist

SUGASRIJAYARAM S T 0 Comments Add Tag

State: Active Area: Music Playlist Project Reason: Regression Iteration: Music Playlist Project\Iteration 1

4/29/2026 5:30 PM Bug filed on "TC06 - Add and Remove Songs from Playlist"

Step no.	Result	Title
1.	Passed	Open an existing playlist Expected Result Playlist details are displayed
2.	Failed	Click "Add Song" Expected Result Song selection options appear
3.	Failed	Select a song Expected Result Song is added to the playlist

System Info

Discussion

Story Points

Priority

Severity

3 - Medium

Activity

Effort (Hours)

Original Estimate

Remaining

Completed

Planning Poker

Voted 0

0 1 2 3 5 8 13

Write your comment here

Details

Item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.

Related Work

Add link

Parent

23 Playlist Creation and Management
Updated 58m ago Resolved

Tested By

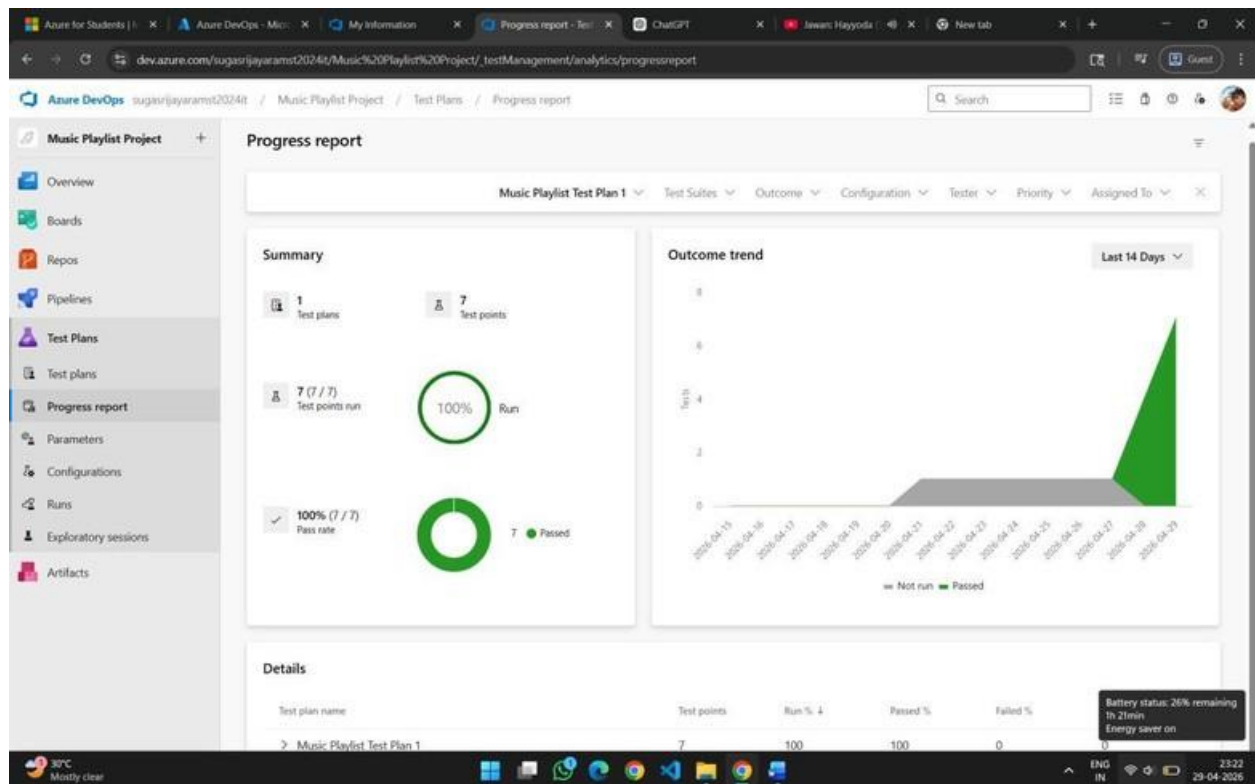
85 TC06 - Add and Remove Songs from Playlist
Updated 1h ago Closed

System Info

Found in Build

Integrated in Build

8. Progress report



Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path.

EXPNO: 9

Create Epic, Features, User Stories, Task

LOAD TESTING AND PIPELINES**Aim:**

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment.

Load Testing**Azure Load Testing:**

Azure Load Testing allows you to simulate high traffic and stress tests for your web applications and APIs to understand how they perform under load. It helps identify performance bottlenecks, scalability issues, and optimize resource usage before deployment.

Steps to Create an Azure Load Testing Resource:

Before you run your first test, you need to create the Azure Load Testing resource:

1. Sign in to Azure Portal
Go to <https://portal.azure.com> and log in.
2. Create the Resource
 - o Go to *Create a resource* → Search for "Azure Load Testing".
 - o Select Azure Load Testing and click Create.
3. Fill in the Configuration Details
 - o *Subscription*: Choose your Azure subscription.
 - o *Resource Group*: Create new or select an existing one.
 - o *Name*: Provide a unique name (no special characters).
 - o *Location*: Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create.
6. Once deployment is complete, click Go to resource.

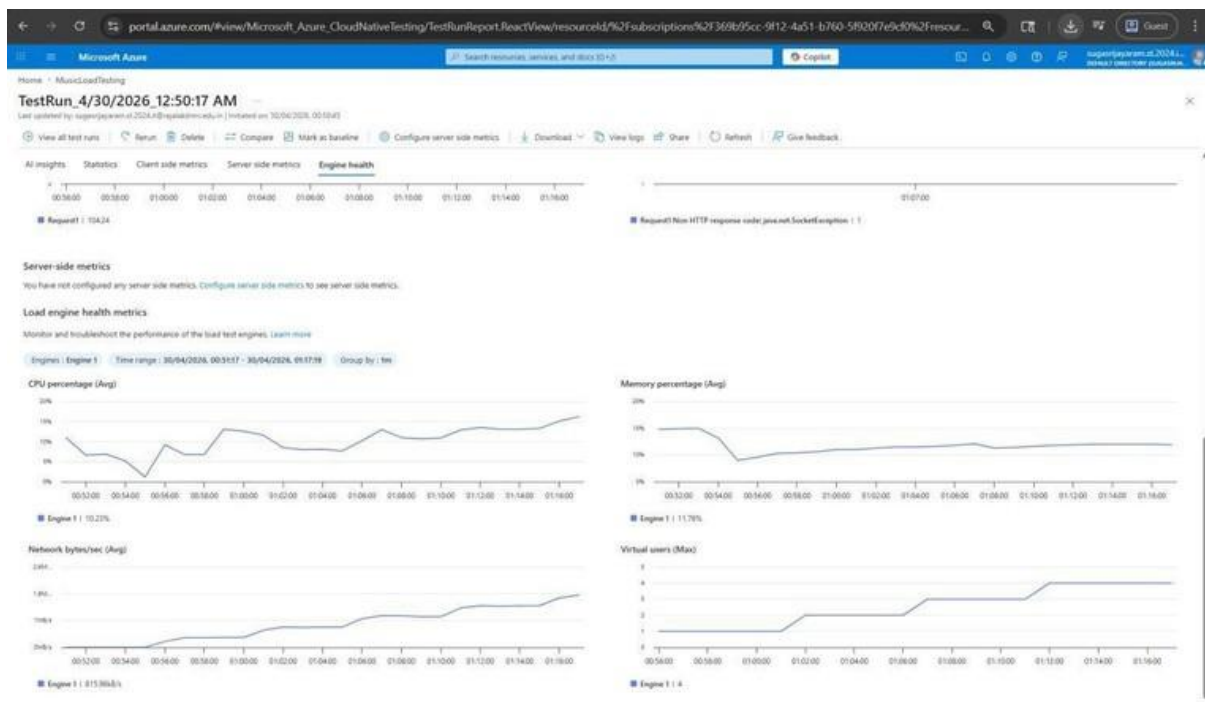
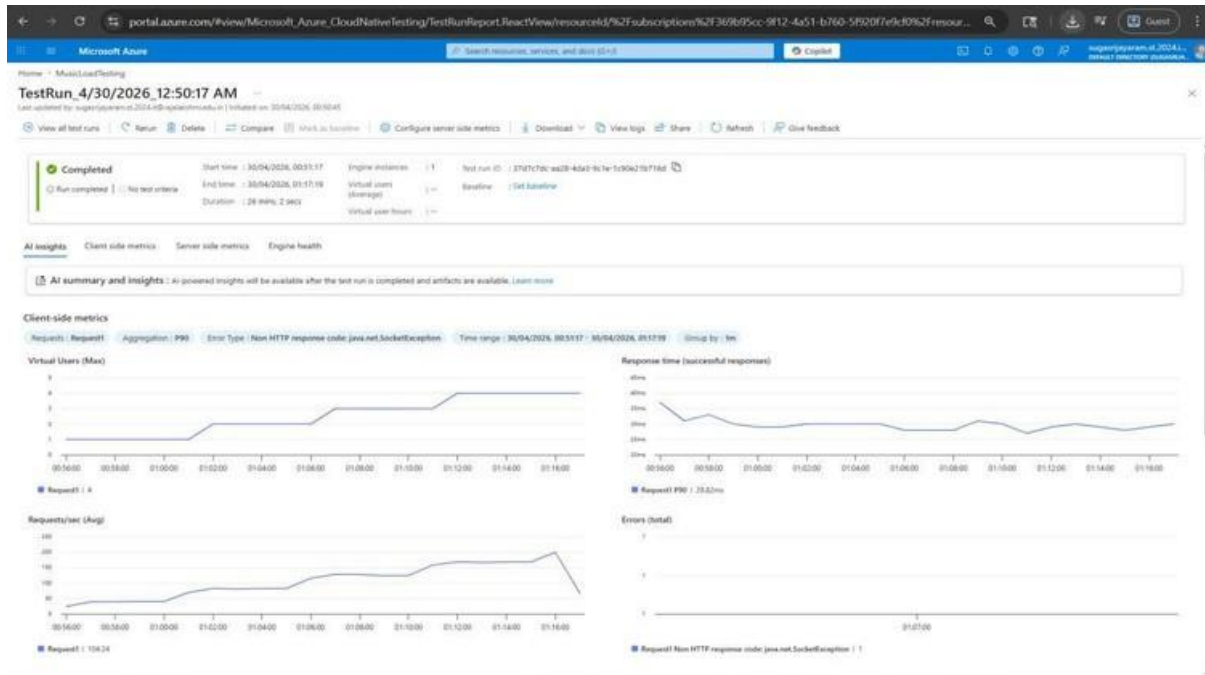
Steps to Create and Run a Load Test:

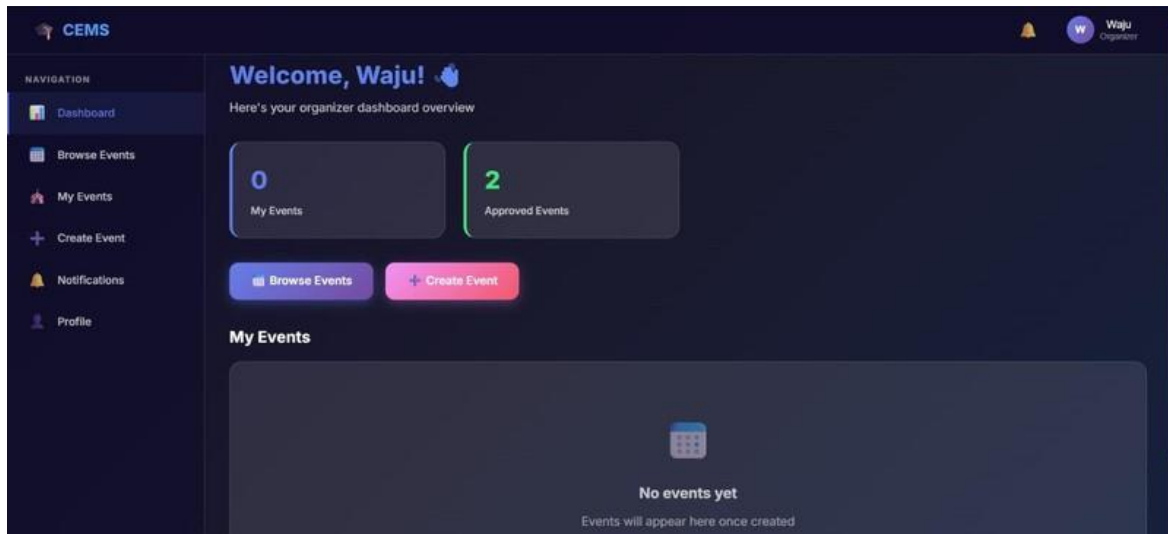
Once your resource is ready:

1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab
 - o *Test Name*: Provide a unique name.
 - o *Description*: (Optional) Add test purpose.
 - o *Run After Creation*: Keep checked.
3. Load Settings
 - o *Test URL*: Enter the target endpoint (e.g., <https://yourapi.com/products>).

4. Click Review + Create → Create to start the test.

Load Testing





Pipelines

Description:

This experiment demonstrates how to connect a GitHub-hosted Flask-based music recommendation project with Azure DevOps. The pipeline will automatically install dependencies, run basic tests, and publish artifacts. This ensures that every commit triggers checks for reliability and smooth deployment.

Steps:

1. Connect GitHub to Azure DevOps:
 - o In Azure DevOps, create a new project.
 - o Create a pipeline and select GitHub as the source.
 - o Authorize access to your GitHub repository, ensuring that Azure DevOps can pull the repository for your pipeline.
2. Create azure-pipelines.yml in Your Repo Root:
 - o In your GitHub repository, create a new file called azure-pipelines.yml in the root directory.
 - o Add the following basic pipeline configuration for Python and Flask:

yaml Code

trigger:

- main

pool:

vmImage: 'ubuntu-latest'

variables:

NODE_VERSION: '20.19.x'

NPM_CACHE: '\${Pipeline.Workspace}/.npm'

steps:

♦ Install Node.js (UPDATED TASK)

- task: UseNode@1

inputs:

version: \$(NODE_VERSION)

displayName: 'Install Node.js'

♦ Cache npm (FIXED PATH)

- task: Cache@2

inputs:

key: 'npm | "\$(Agent.OS)" | **/package-lock.json'

path: \$(NPM_CACHE)

displayName: 'Cache npm'

♦ Install Frontend Dependencies

- script: |

npm config set cache \$(NPM_CACHE) --global

npm ci

workingDirectory: frontend

displayName: 'Install Frontend Dependencies'

♦ Build Frontend (Vite)

- script: npm run build

workingDirectory: frontend

displayName: 'Build Frontend'

♦ Frontend Tests (optional)

```
- script: npm test --if-present
  workingDirectory: frontend
  displayName: 'Run Frontend Tests'
```

♦ Install Backend Dependencies

```
- script: |
  npm config set cache $(NPM_CACHE) --global
  npm ci
  workingDirectory: backend
  displayName: 'Install Backend Dependencies'
```

♦ Backend Tests

```
- script: npm test --if-present
  workingDirectory: backend
  displayName: 'Run Backend Tests'
```

♦ Verify Frontend Build (fail early)

```
- script: |
  if [ ! -d "frontend/dist" ]; then
    echo "Build failed: dist folder not found"
    exit 1
  fi
  displayName: 'Verify Frontend Build'
```

♦ Archive Frontend

```
- task: ArchiveFiles@2
  inputs:
    rootFolderOrFile: 'frontend/dist'
    includeRootFolder: false
    archiveType: 'zip'
    archiveFile: '$(Build.ArtifactStagingDirectory)/frontend.zip'
    displayName: 'Archive Frontend'
```

♦ Archive Backend

```
- task: ArchiveFiles@2
  inputs:
    rootFolderOrFile: 'backend'
    includeRootFolder: false
    archiveType: 'zip'
    archiveFile: '$(Build.ArtifactStagingDirectory)/backend.zip'
    displayName: 'Archive Backend'
```

♦ Publish Artifacts

- task: PublishBuildArtifacts@1

inputs:

PathToPublish: '\$(Build.ArtifactStagingDirectory)'

ArtifactName: 'drop'

displayName: 'Publish Artifacts'

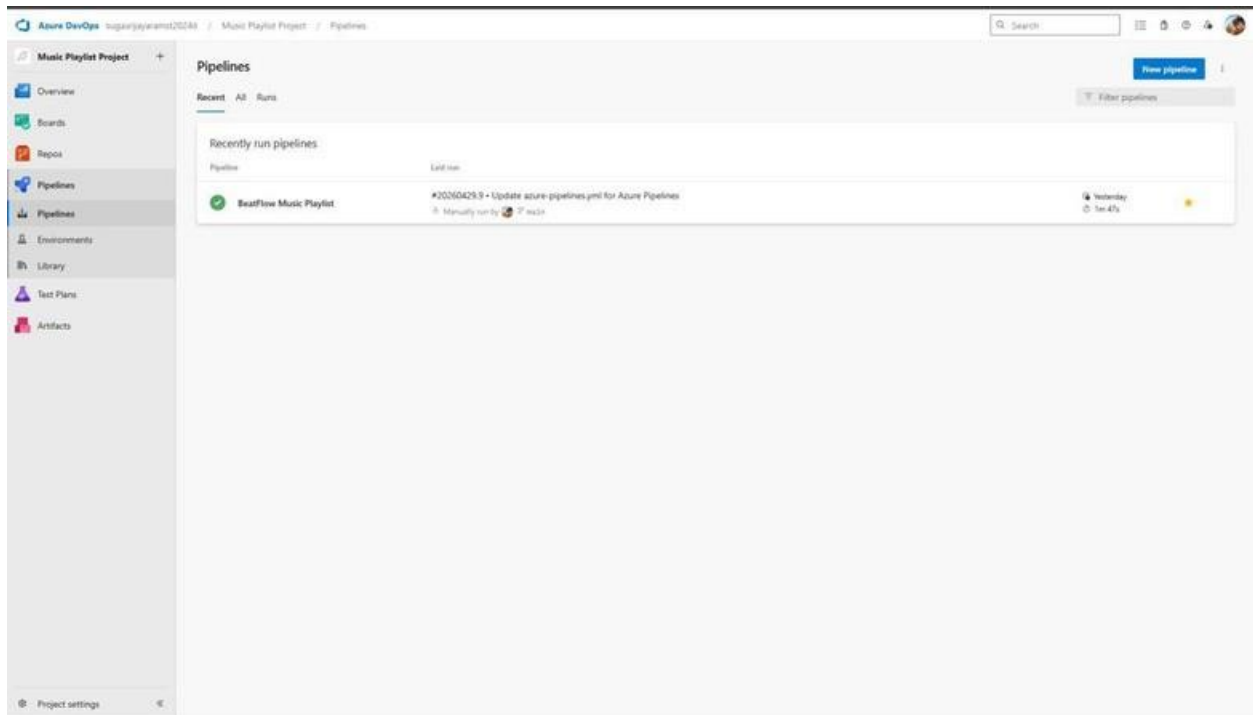
3. Run and Monitor Pipeline:

- o Commit changes to the main branch of your repository to trigger the pipeline in Azure DevOps.
- o Monitor the logs in the Azure DevOps portal to view logs, errors, or success messages and ensure everything runs smoothly.

Pipeline

The screenshot shows the Azure DevOps web portal interface for reviewing a pipeline YAML file. The sidebar on the left contains navigation links for Overview, Boards, Repos, Pipelines, Environments, Library, Test Plans, and Artifacts. The main content area is titled 'Review your pipeline YAML' and displays the 'azure-pipelines.yml' file. The pipeline configuration includes a trigger for the 'main' branch, a pool using 'ubuntu-latest', and several steps: installing Node.js, caching npm, installing frontend dependencies, and building the frontend with Vite. The interface also includes a 'Variables' section and a 'Save and run' button.

```
1 # Starter pipeline
2 # Start with a minimal pipeline that you can customize to build and deploy your code.
3 # Add steps that build, run tests, deploy, and more:
4 # https://aka.ms/yaml
5
6 trigger:
7   - main
8
9 pool:
10  vmImage: 'ubuntu-latest'
11
12 variables:
13   NODE_VERSION: '20.19.x'
14   NPM_CACHE: '$(Pipeline.Workspace)/.npm'
15
16 steps:
17
18   # - Install Node.js (UPDATED TASK)
19   - task: UseNodejs1
20     inputs:
21       version: '$(NODE_VERSION)'
22       displayName: 'Install Node.js'
23
24   # - Cache npm (FIXED PATH)
25   - task: Cache@2
26     inputs:
27       key: 'npm | "$(Agent.OS)" | **/package-lock.json'
28       path: '$(NPM_CACHE)'
29       displayName: 'Cache npm'
30
31   # - Install Frontend Dependencies
32   - script: |
33       npm config set cache $(NPM_CACHE) --global
34       npm ci
35     workingDirectory: frontend
36     displayName: 'Install Frontend Dependencies'
37
38   # - Build Frontend (Vite)
39   - script: npm run build
40     workingDirectory: frontend
41     displayName: 'Build Frontend'
```



Result:

Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops.

EXPNO: 10

Create Epic, Features, User Stories, Task

GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the College Event Management.

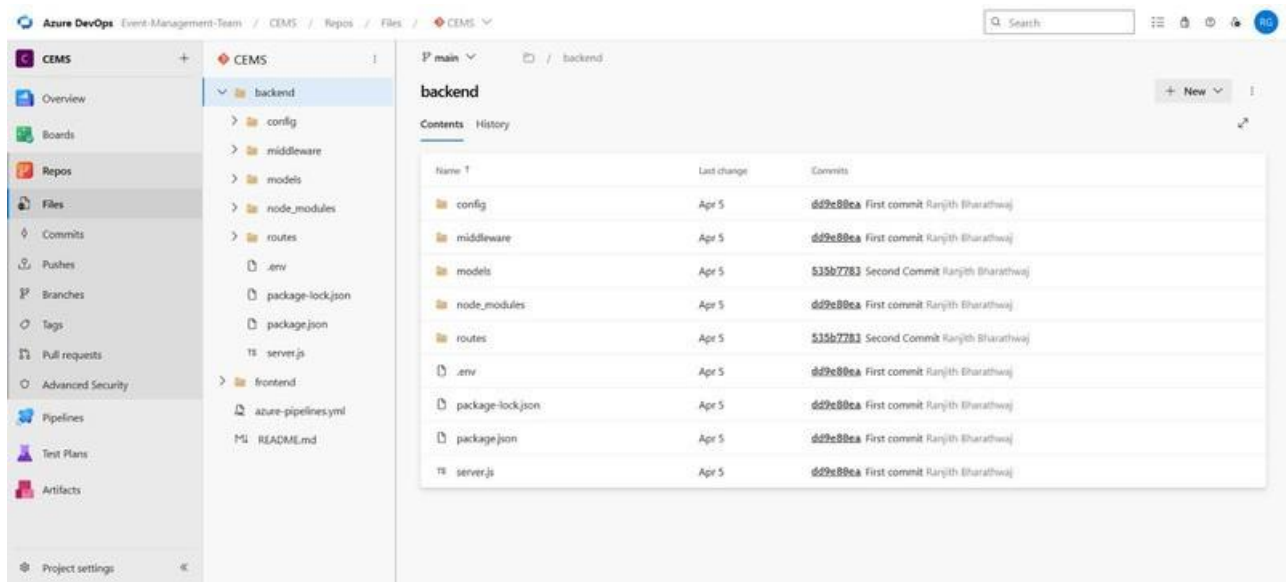
GitHub Project Structure

The screenshot displays the GitHub interface for a repository named 'CEMS'. The left sidebar shows the repository's navigation menu, including Overview, Boards, Repos, Files, Commits, Pushes, Branches, Tags, Pull requests, Advanced Security, Pipelines, Test Plans, and Artifacts. The main content area shows the 'Files' tab, which lists the repository's structure:

Name	Last change	Commits
backend	Apr 5	535b7783 Second Commit Ranjith Bharathwaj
frontend	Apr 5	6d9e88ea First commit Ranjith Bharathwaj
azure-pipelines.yml	Apr 5	6d9e88ea First commit Ranjith Bharathwaj
README.md	Apr 5	6d9e88ea First commit Ranjith Bharathwaj

Below the file list, the project description for 'College Event Management System (CEMS)' is shown, stating it is a full-stack web application for managing college events built with React, Node.js/Express, and MongoDB. The 'Features' section lists the following:

- User Management** — Registration, login, profile management with JWT auth
- Role-Based Access** — Student, Organizer, Admin roles with different permissions
- Event Management** — Create, edit, delete events with admin approval workflow
- Event Discovery** — Browse, search, and filter events by category
- Registration System** — Register for events with duplicate prevention
- Notifications** — Real-time notifications for registrations and event updates
- Admin Dashboard** — Reports, user management, event approvals



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.